

Matteo Palmonari

matteo.palmonari@disco.unimib.it

2.4 *Querying RDF Knowledge Graphs* *SPARQL*



dat•AI data * AI
AI * data

Reasoning with RDF Knowledge Graphs: Outline

■ RDF Knowledge Graphs

- Data model: RDF

- **Query language: SPARQL**

- Vocabularies and ontologies

- Basic reasoning: RDFS

- Advanced reasoning: OWL2

DATA

KNOWLEDGE
ENGINEERING

MODELING

REASONING

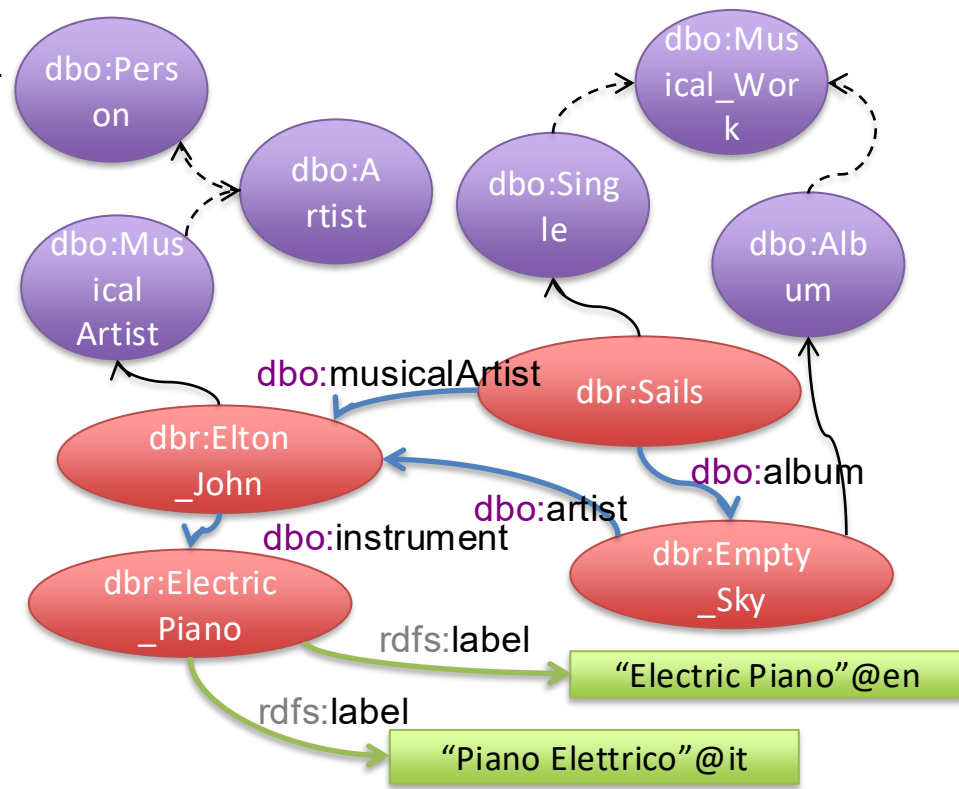
RDF: IRIfied Graphs and Triples

Proper
Turtle Syntax

@prefix dbr: <http://dbpedia.org/resource/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix rdfs: <https://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

dbr:Electric_Piano rdfs:label "Electric Piano"@en .
dbr:Electric_Piano rdfs:label "Piano Elettrico"@it .
dbr:Elton_John dbo:instrument dbr:Electric_Piano .
dbr:Empty_Sky dbo:artist dbr:Elton_John .
dbr:Sails dbo:musicalArtist dbr:Elton_John .
dbr:Sails dbo:album dbr:Empty_Sky .
dbr:Elton_John dbo:instrument dbr:Electric_Piano .
dbr:Elton_John rdf:type dbr:Musical_Artist .
dbr:Sails rdf:type dbo:Single .
dbr:Empty_Sky rdf:type dbo:Album .
dbo:Musical_Artist rdfs:subClassOf dbo:Artist .
dbo:Artist rdfs:subClassOf dbo:Person .
dbo:Album rdfs:subClassOf dbo:Musical_Work .
dbo:Single rdfs:subClassOf dbo:Musical_Work .

In proper Turtle syntax: with prefixes





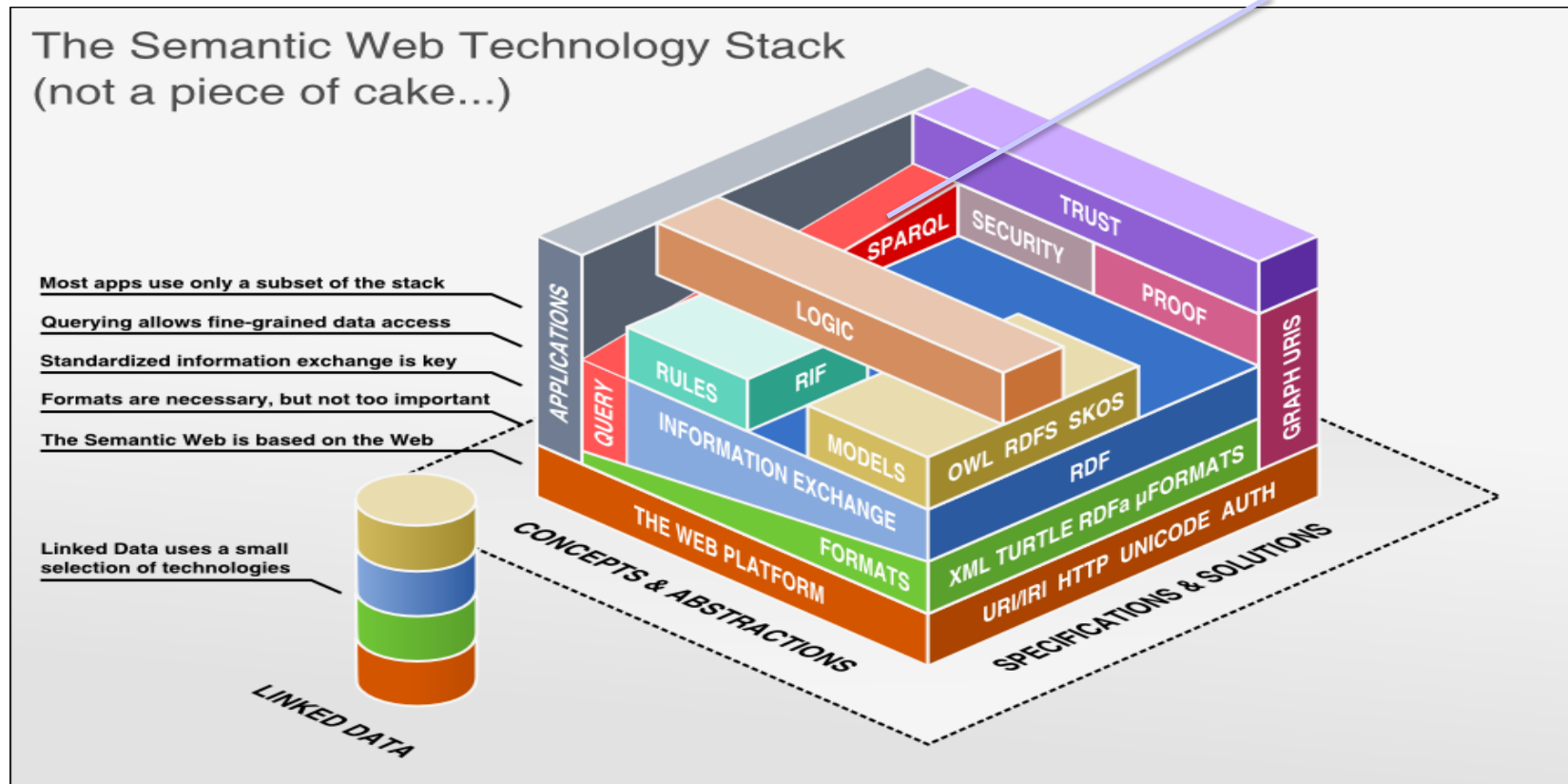
SPARQL

Query RDF data

01 – Introduction to SPARQL

Semantic Web Stack

A Declarative query language for RDF



<http://bnode.org/blog/2009/07/08/the-semantic-web-not-a-piece-of-cake>

Outline of These Lessons (2)

- Introduction
 - What is SPARQL
- Basic Query Patterns (Tabular Results)
 - Idea
 - Semantics
- Graph Traversal Queries (Graph Queries)
 - Idea
 - Semantics
- Query Forms, Modifiers, Reasoning, etc.

- **SPARQL Query**
 - Declarative query language for RDF data
 - <http://www.w3.org/TR/rdf-sparql-query/>
- **SPARQL Algebra**
 - Mapping from queries to relational algebra for result manipulation
 - <http://www.w3.org/2001/sw/DataAccess/rq23/rq24-algebra.html>
- **SPARQL Update**
 - Declarative manipulation language for RDF data
 - <http://www.w3.org/TR/sparql11-update/>
- **SPARQL Protocol**
 - Standard for communication between SPARQL services and clients
 - <http://www.w3.org/TR/sparql11-protocol/>

SPARQL Query 1.1

- **SPARQL 1.0** only allows accessing the data (**query**)
- **SPARQL 1.1** introduces:

Query extensions



- Aggregates, Subqueries, Negation, Expressions in the SELECT clause, **Property paths**, assignment, short form for CONSTRUCT, expanded set of functions and operators

Updates



- **Data management:** Insert, Delete, Delete/Insert
- **Graph management:** Create, Load, Clear, Drop, Copy, Move, Add

Federation extension



- Service, values, service variables (informative)

Scope of this Presentation and Additional Online Resources

- Scope of this presentation
 - Explanation of key principles and features of SPARQL (shortened and modified version of EUCLID slides – see below)
 - Precise semantics of query evaluation
- Additional details on SPARQL available online:
 - SPARQL specification:
 - <https://www.w3.org/TR/sparql11-overview/> (overview of SPARQL)
 - <https://www.w3.org/TR/sparql11-query/> (language specification)
 - EUCLID Module 2 - SPARQL: Querying Linked Data
 - Available at: <https://www.slideshare.net/EUCLIDproject/querying-linked-data>
 - EUCLID project: training for semantic technologies, 6 modules, slides available with CC license at <http://euclid-project.eu/>

How to Get Familiar with SPARQL

- Get familiar with RDF data by browsing DBpedia online
 - E.g., search for “Elton John dbpedia” (or a singer you like better), jump to its page, follow the links and explore more data; check in particular properties and types (for the moment, types are values of the property `rdf:type`; we will further discuss types in the next lessons)
- Understand the key principles underpinning SPARQL
 - This presentation
- Make exercises
 - SPARQL hands-on session in this course (next lesson)
 - Make queries to the DBpedia SPARQL portal at <http://dbpedia.org/sparql>
 - While making queries use the SPARQL specification as a reference:
 - <https://www.w3.org/TR/sparql11-query/> (language specification)



SPARQL

Query RDF data

02 –SPARQL Basics
and Query Components

SPARQL Basics #1

- **RDF graph:** Set of RDF assertions, manipulated as a labeled directed graph.
- **RDF data set:** set of RDF triples. It is comprised of:
 - One default graph
 - Zero or more named graphs
- **SPARQL protocol client:** HTTP client that sends requests for SPARQL Protocol operations (queries or updates)
- **SPARQL protocol service:** HTTP server that services requests for SPARQL Protocol operations
- **SPARQL endpoint:** The IRI at which a SPARQL Protocol service listens for requests from SPARQL clients

SPARQL Basics / KGBook

RDF data set:

- One default graph
- Zero or more named graphs

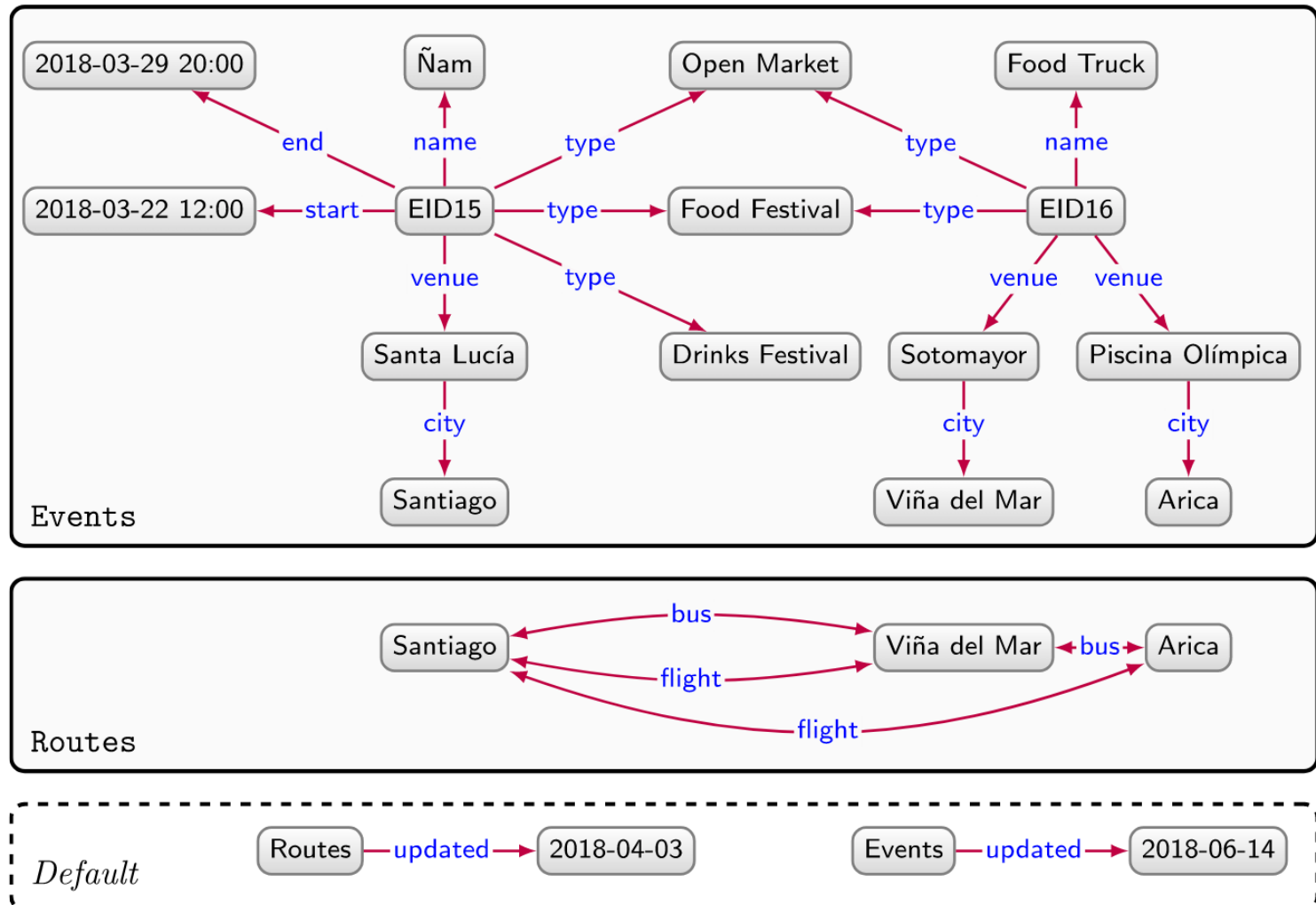


Figure 2.4: Graph dataset based on directed edge-labelled graphs with two named graphs and a default graph describing events and routes

SPARQL Basics #1

- **RDF graph:** Set of RDF assertions, manipulated as a labeled directed graph.
- **RDF data set:** set of RDF triples. It is comprised of:
 - One default graph
 - Zero or more named graphs
- **SPARQL protocol client:** HTTP client that sends requests for SPARQL Protocol operations (queries or updates)
- **SPARQL protocol service:** HTTP server that services requests for SPARQL Protocol operations
- **SPARQL endpoint:** The IRI at which a SPARQL Protocol service listens for requests from SPARQL clients

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>

SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

Prologue:

- Prefix definitions
- Subtly different from Turtle syntax - the final period is not used

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>  
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
PREFIX dc: <http://purl.org/dc/elements/1.1/>  
PREFIX mo: <http://purl.org/ontology/mo/>
```

```
SELECT ?album  
FROM <http://musicbrainz.org/20130302>  
WHERE {  
    dbpedia:The_Beatles foaf:made ?album .  
    ?album a mo:Record ; dc:title ?title  
}  
ORDER BY ?title
```

More on forms at the
end of the presentation

Query form:

- ASK, SELECT, DESCRIBE or CONSTRUCT
- SELECT retrieves variables and their bindings as a table

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>  
PREFIX foaf: <http://xmlns.com/foaf/0.1/>  
PREFIX dc: <http://purl.org/dc/elements/1.1/>  
PREFIX mo: <http://purl.org/ontology/mo/>
```

```
SELECT ?album  
FROM <http://musicbrainz.org/20130302>  
WHERE {  
    dbpedia:The_Beatles foaf:made ?album .  
    ?album a mo:Record ; dc:title ?title  
}  
ORDER BY ?title
```

Graph names

Data set specification:

- This clause is optional
- FROM or FROM NAMED
- Indicates the sources for the data against which to find matches

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>

SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

Data set specification:

- This clause is optional
- FROM or FROM NAMED
- Indicates the sources for the data against which to find matches

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>

SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

Query pattern:

- Defines patterns to match against the data
- Generalises Turtle with variables and keywords – N.B. final period optional

Remember Turtle abbreviations

- Same subject:
 - `<[...]> a mo:Record ; dc:title "Help!" .` ... equivalent to:
 - `<[...]> a mo:Record .`
 - `<[...]> dc:title "Help" .`
- Same subject and predicate:
 - `<[...]> dc:title "Help!@En" , "Aiuto!@It" .` ... equivalent to:
 - `<[...]> dc:title "Help!@En" .`
 - `<[...]> dc:title "Aiuto!@It" .`

```
WHERE {  
    dbpedia:The_Beatles foaf:made ?album .  
    ?album a mo:Record ; dc:title ?title  
}  
ORDER BY ?title
```

Query pattern:

- Defines patterns to match against the data
- Generalises Turtle with variables and keywords – N.B. final period optional

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>

SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

Query pattern:

- Defines patterns to match against the data
- Generalises Turtle with variables and keywords – N.B. final period optional

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>

SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

Check SPARQL specifications when you need to use these advanced features. Once you understood WHERE clauses they are quite straightforward

Solution modifier:

- Modify the result set
- ORDER BY, LIMIT or OFFSET re-organise rows;
- GROUP BY combines them

SPARQL Query: Components

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>

SELECT ?album
FROM <http://musicbrainz.org/20130302>
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album a mo:Record ; dc:title ?title
}
ORDER BY ?title
```

Query pattern:

- Defines patterns to match against the data
- Generalises Turtle with variables and keywords – N.B. final period optional

SPARQL Basics #2

- **RDF triple:** Basic building block, of the form subject, predicate, object. Example:

```
dbpedia:The_Beatles foaf:name "The Beatles" .
```

- **RDF triple pattern:** Contains one or more variables. Examples:

```
dbpedia:The_Beatles foaf:made ?album.  
?album mo:track ?track .  
?album ?p ?o .
```

- **RDF quad pattern:** Contains graph name: IRI or variable. Examples:

```
GRAPH <:g> {s :p :o .}  
GRAPH ?g {dbpedia:The_Beatles foaf:name ?o.}
```


SPARQL Query

Main idea: **Pattern matching**

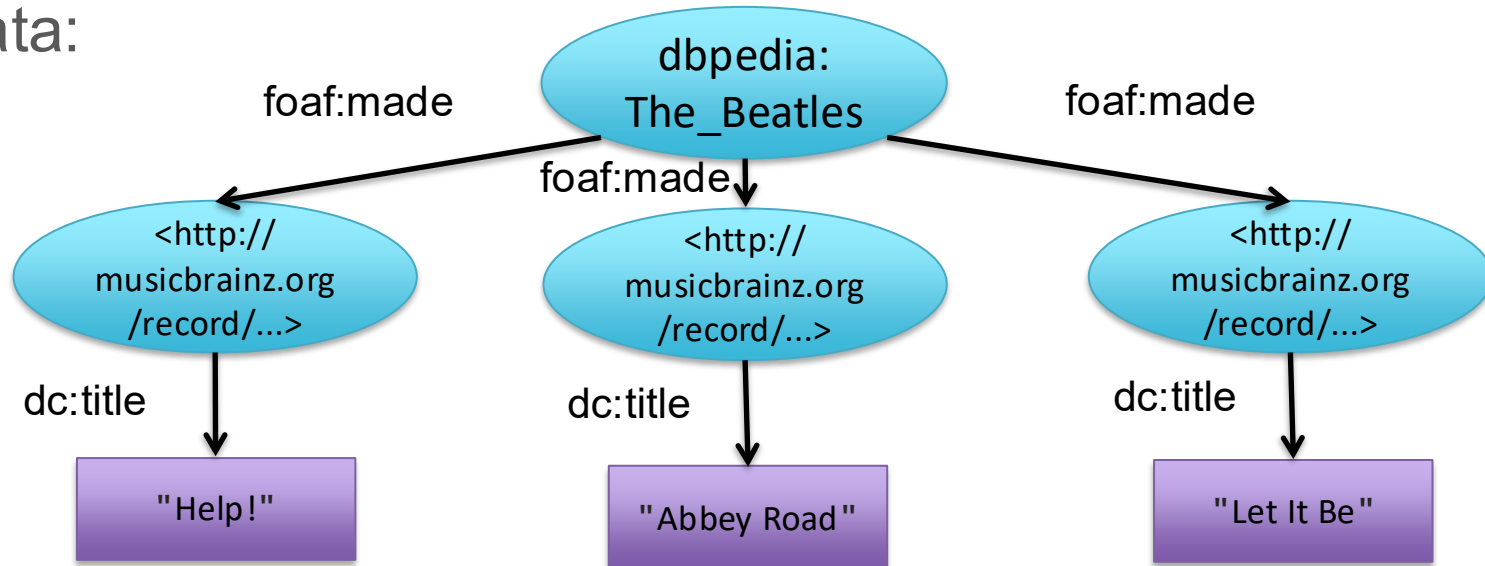
- Queries describe sub-graphs of the queried graph
- **Graph patterns** are RDF graphs specified in Turtle syntax, which contain variables (prefixed by either “?” or “\$”)



- Sub-graphs that match the graph patterns yield a **result**

SPARQL Query

Data:



Results:

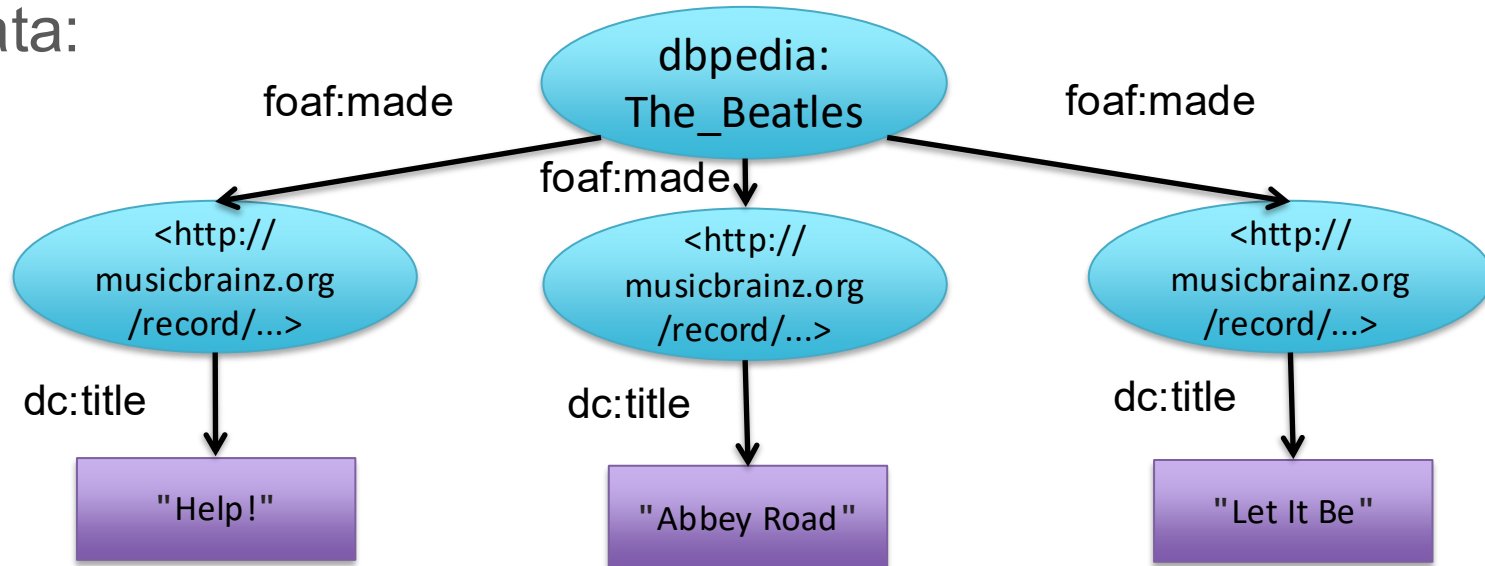
?album

Graph pattern:



SPARQL Query

Data:



Graph pattern:



Results:

?album

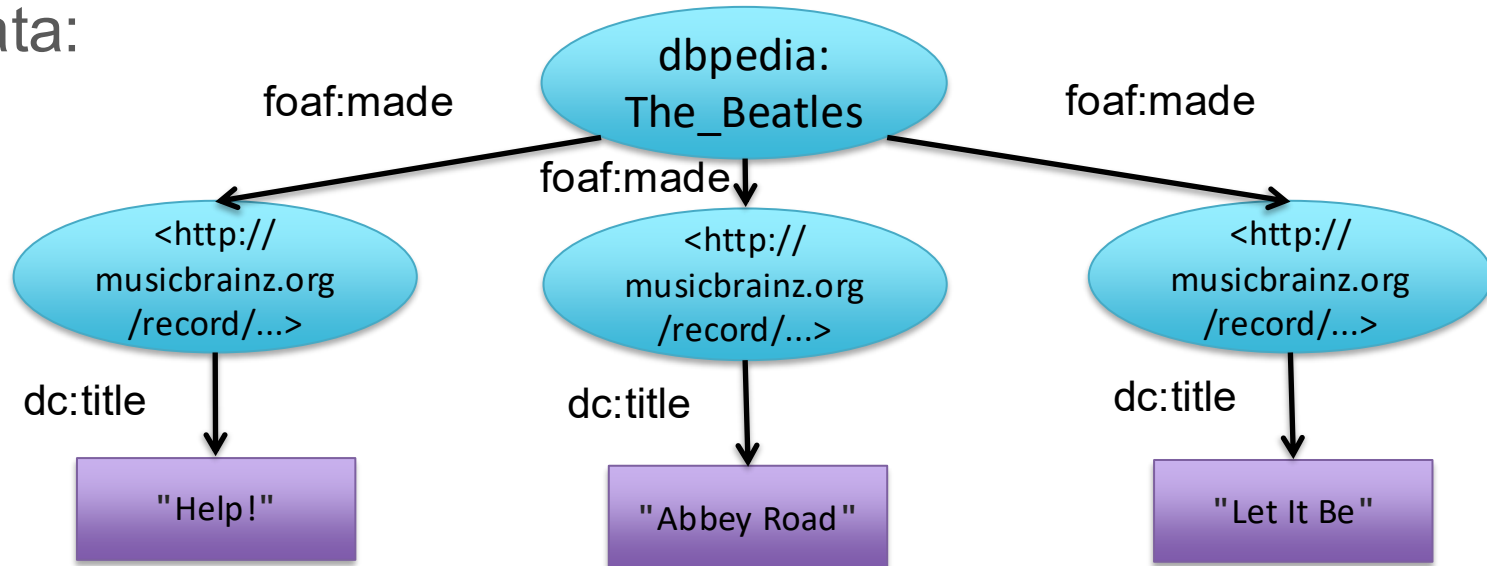
<http://musicbrainz.org/...>

<http://musicbrainz.org/...>

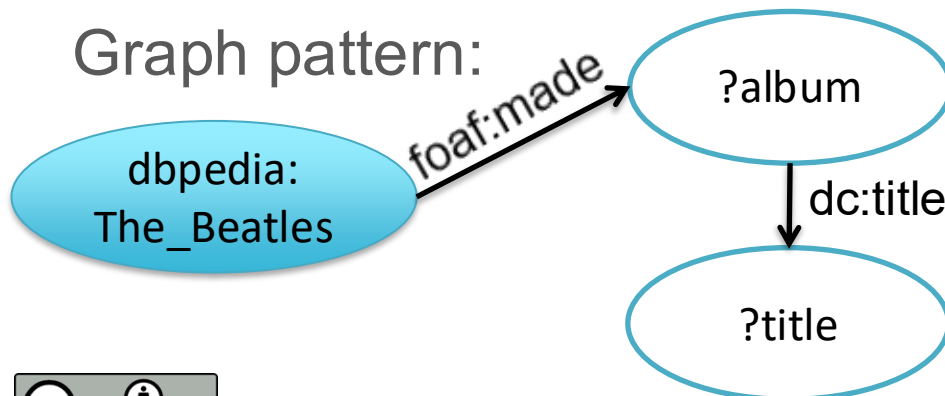
<http://musicbrainz.org/...>

SPARQL Query

Data:



Graph pattern:

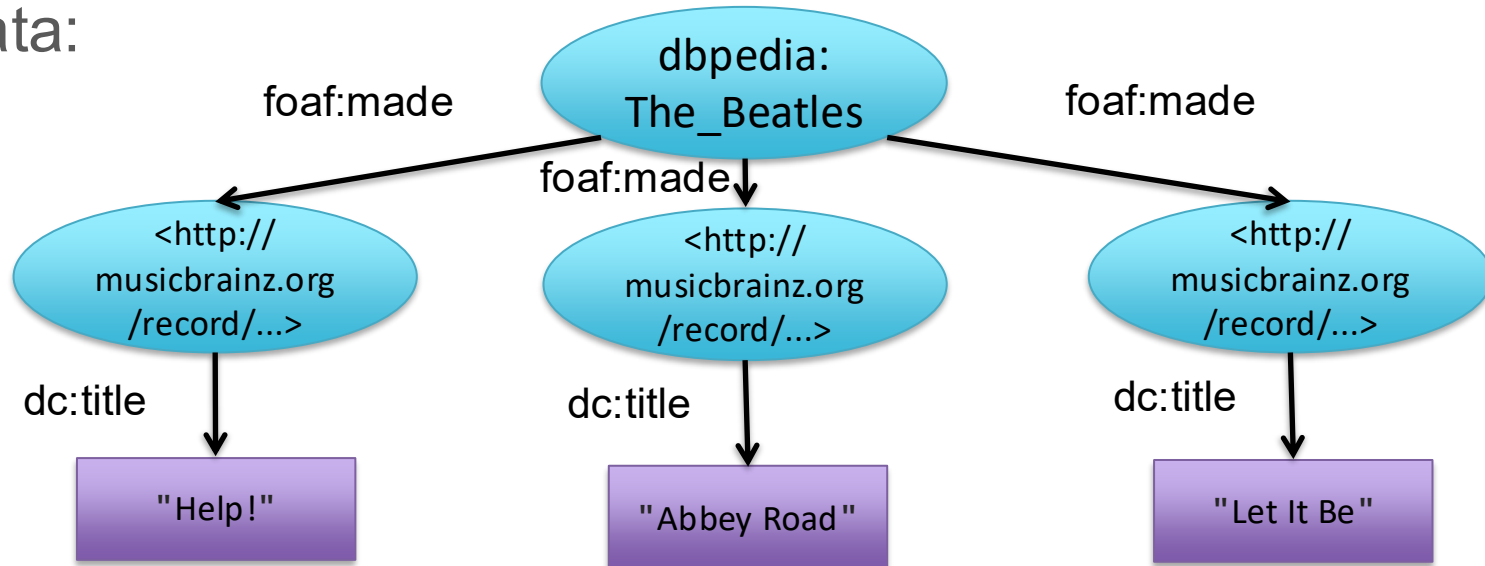


Results:

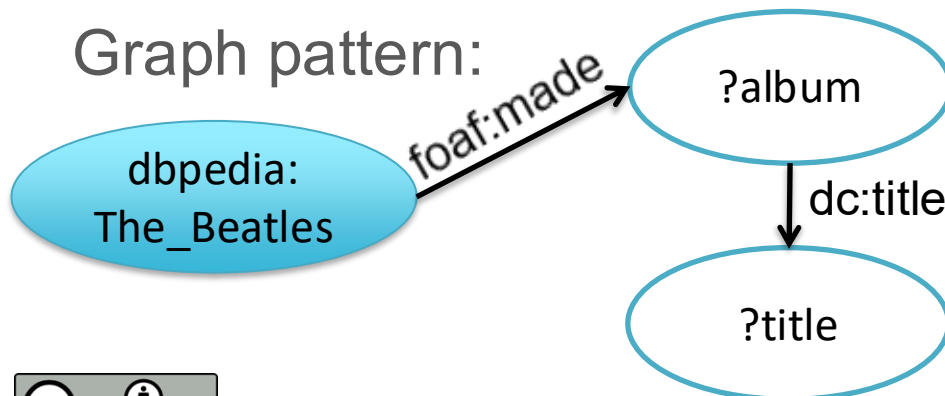
?album	?title
--------	--------

SPARQL Query

Data:



Graph pattern:

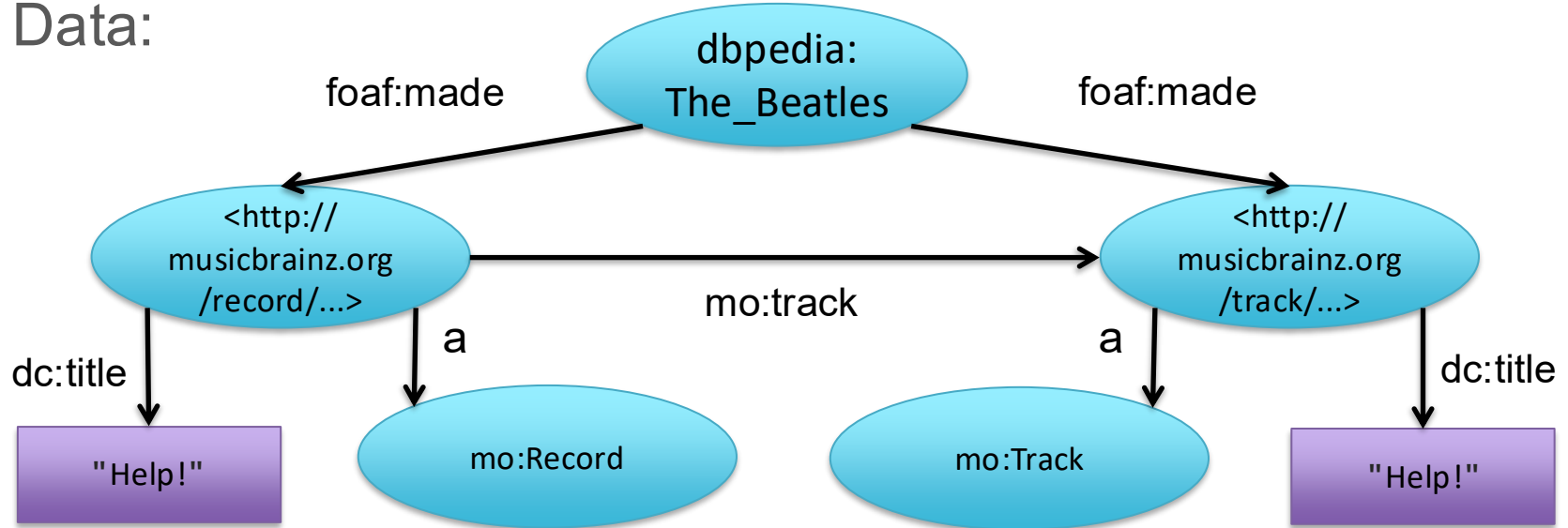


Results:

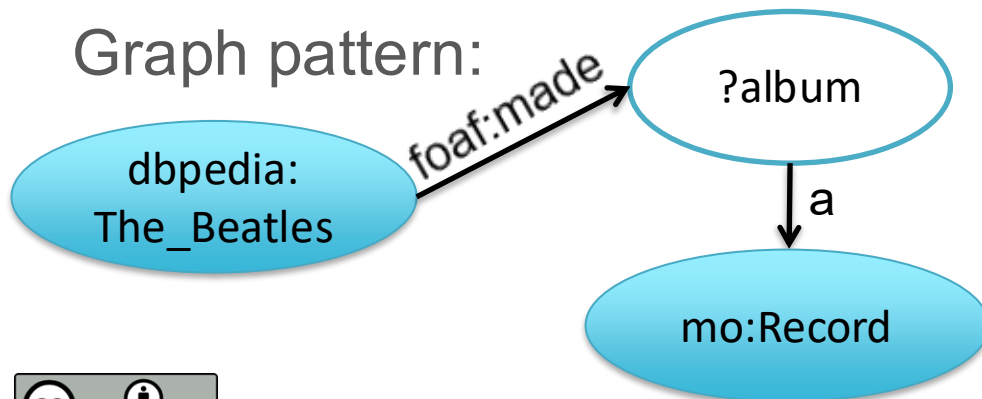
?album	?title
<http://...>	"Help!"
<http://...>	"Abbey Road"
<http://...>	"Let It Be"

SPARQL Query

Data:



Graph pattern:

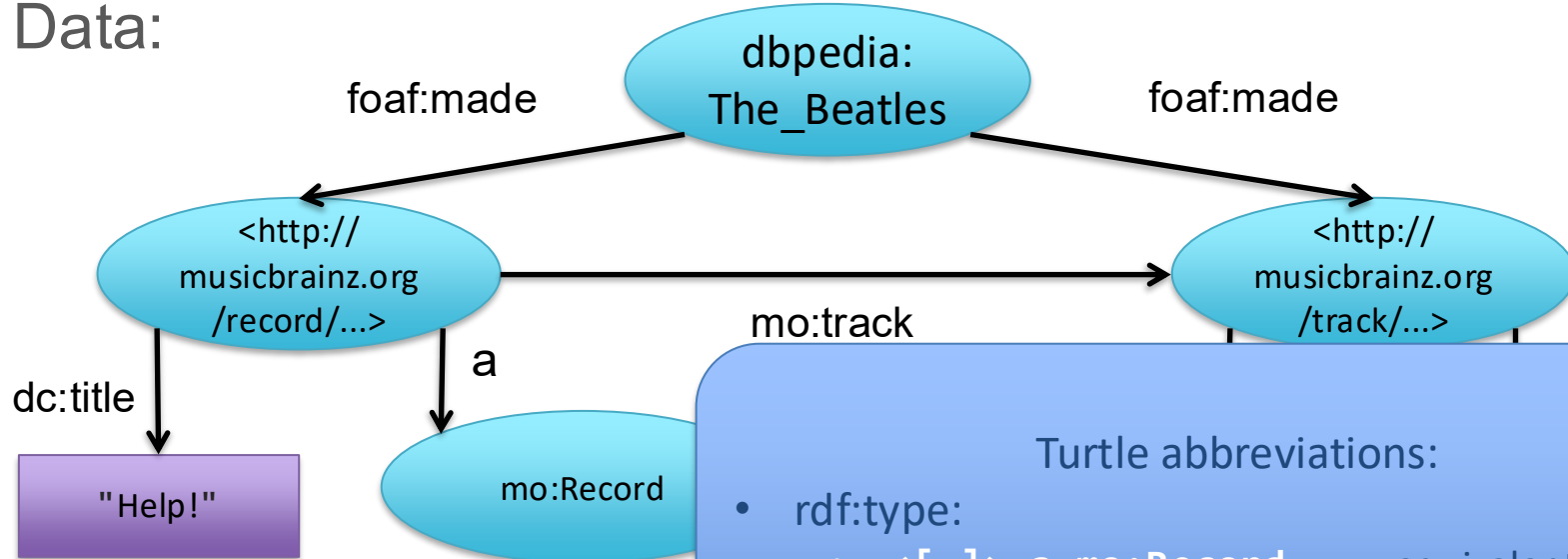


Results:

?album

SPARQL Query

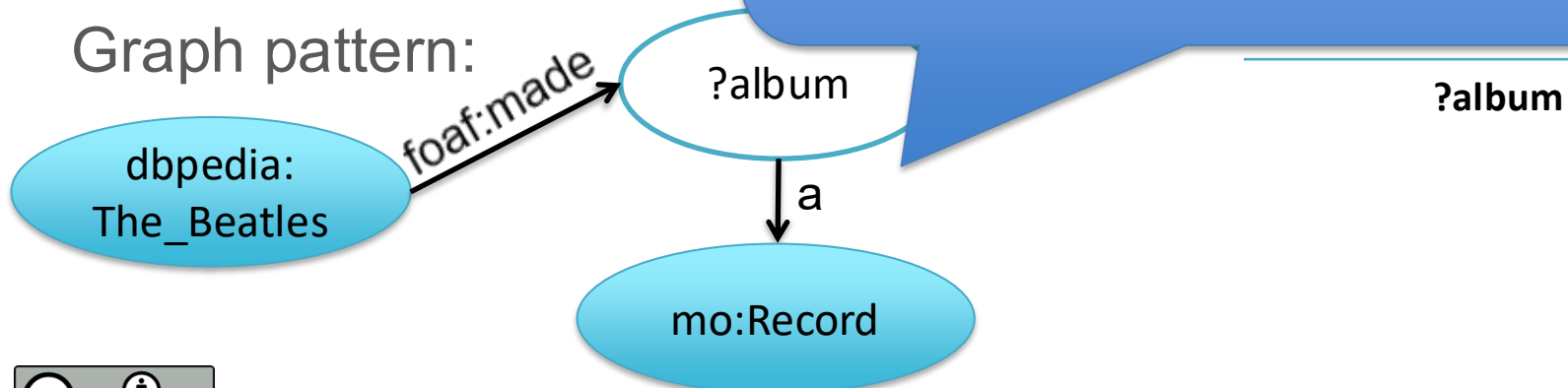
Data:



Turtle abbreviations:

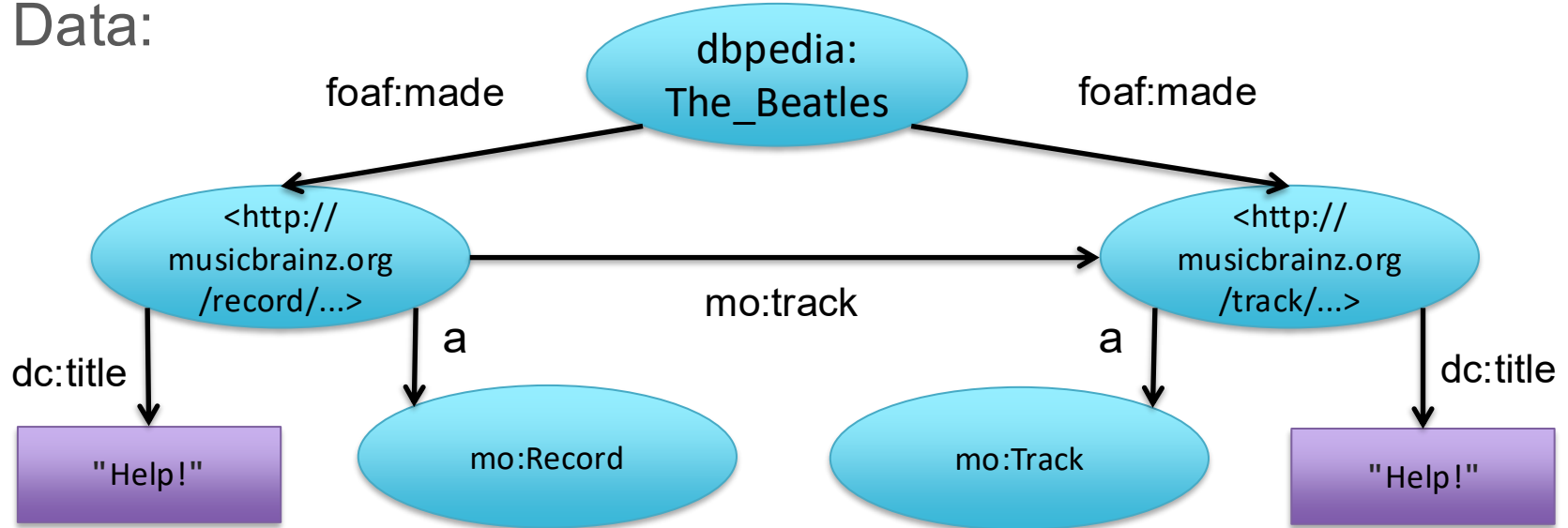
- `rdf:type:`
 - `<[...]> a mo:Record .` ... equivalent to:
 - `<[...]> rdf:type mo:Record .`

Graph pattern:

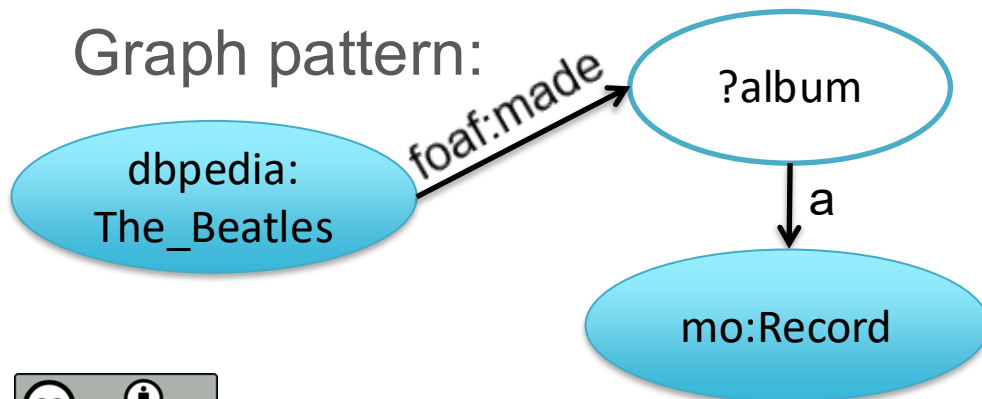


SPARQL Query

Data:



Graph pattern:



Results:

?album

[<http://musicbrainz.org...>](http://musicbrainz.org...)



SPARQL

Query RDF data

03 – Semantics:
Evaluation of Query Patterns

Structure

- Graph patterns
 - Basic graph patterns
 - Complex graph patterns
- Path queries
 - Path expressions
 - Path queries (with variables)
- Mixing graph patterns and path queries
 - Navigational graph queries

Remark on This Lesson

- I go through/share the full presentation, which explains most relevant definitions in the book
- I'll skip some details in the oral presentation and focus on key parts to understand query evaluation (precisely)
- Assumption: you know relational algebra

Graphs vs Graph Patterns

- **Con** = countably infinite set of constants
- **Var** = countably infinite set of variables
- **Term** = **Con** $\cup$ **Var**
- **Con** $\cap$ **Var** = $\{\}$

All definitions* here from KGBook:

<https://kgbook.org/#ssec-querying>

* Some definition has been recently updated, e.g., Directed edge-labeled graph; I keep the previous simpler definitions as they cover properly what we need

Definition 2.1 (Directed edge-labelled graph)

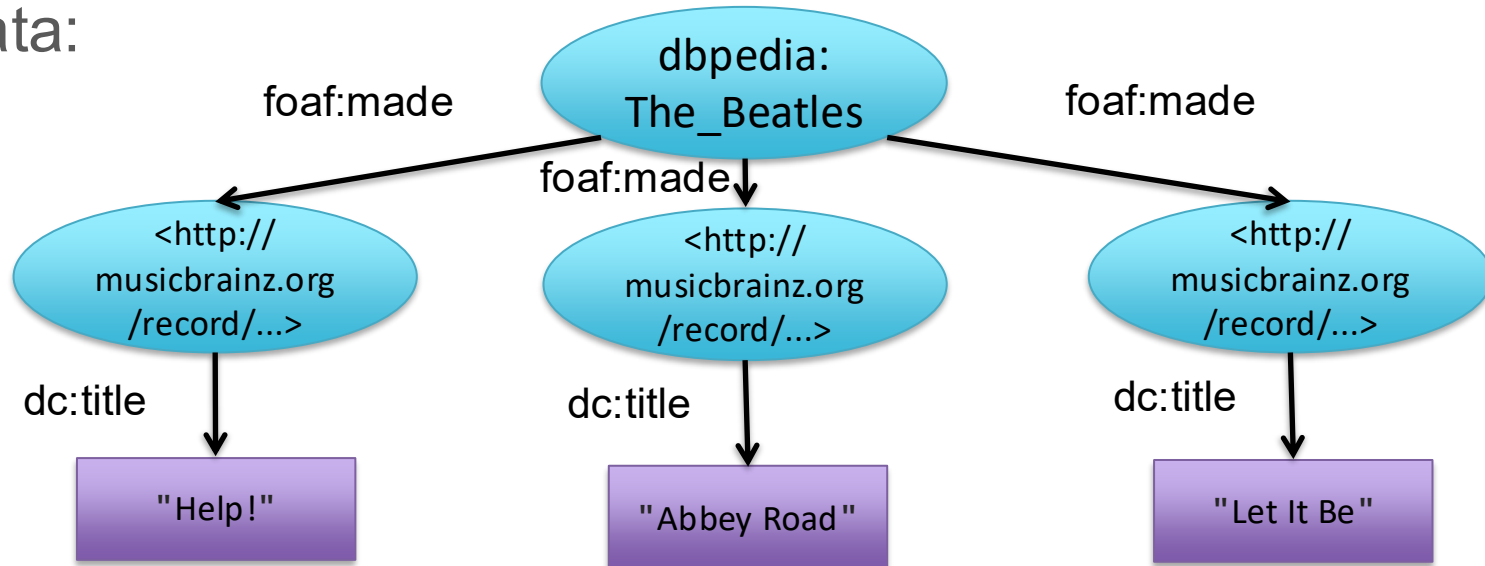
A directed edge-labelled graph is a tuple $G = (V, E, L)$, where $V \subseteq \mathbf{Con}$ is a set of nodes, $L \subseteq \mathbf{Con}$ is a set of edge labels, and $E \subseteq V \times L \times V$ is a set of edges.

Definition 2.5 (Basic directed edge-labelled graph pattern)

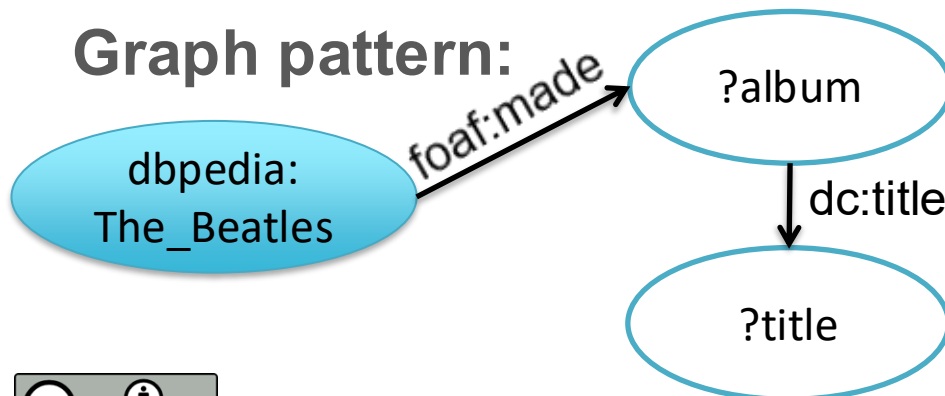
We define a basic directed edge-labelled graph pattern as a tuple $Q = (V, E, L)$, where $V \subseteq \mathbf{Term}$ is a set of node terms, $L \subseteq \mathbf{Term}$ is a set of edge terms, and $E \subseteq V \times L \times V$ is a set of edges (triple patterns).

SPARQL Query

Data:



Graph pattern:

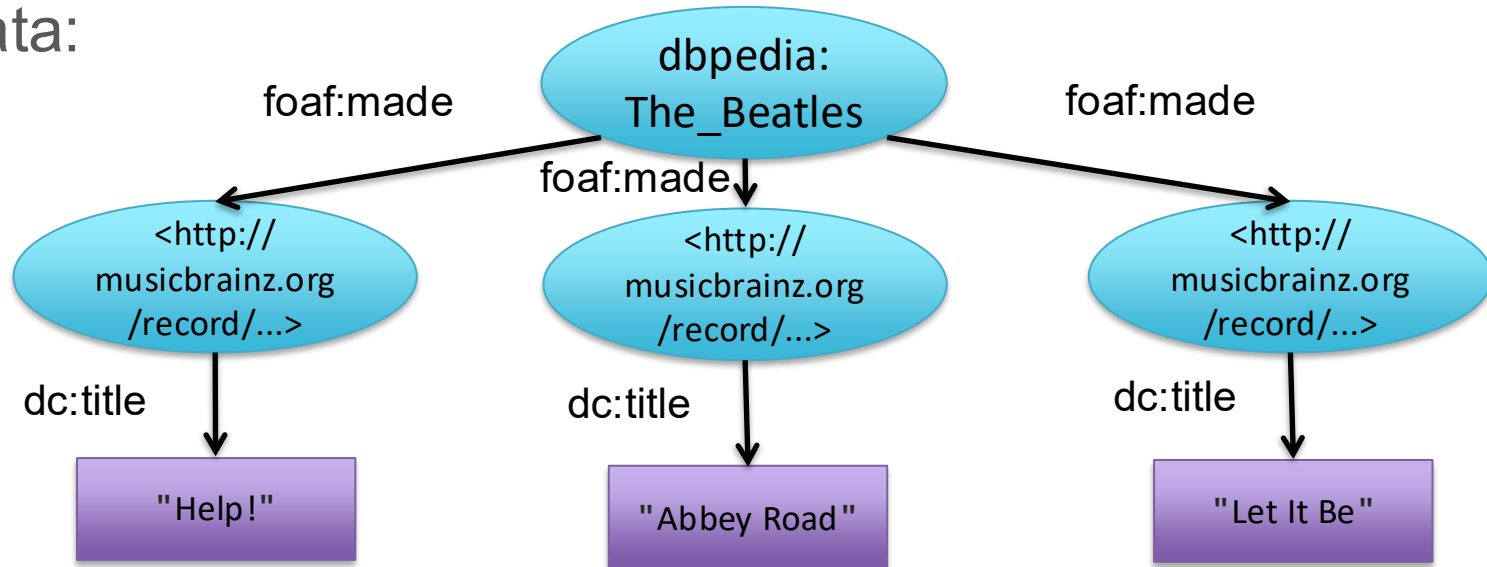


Results:

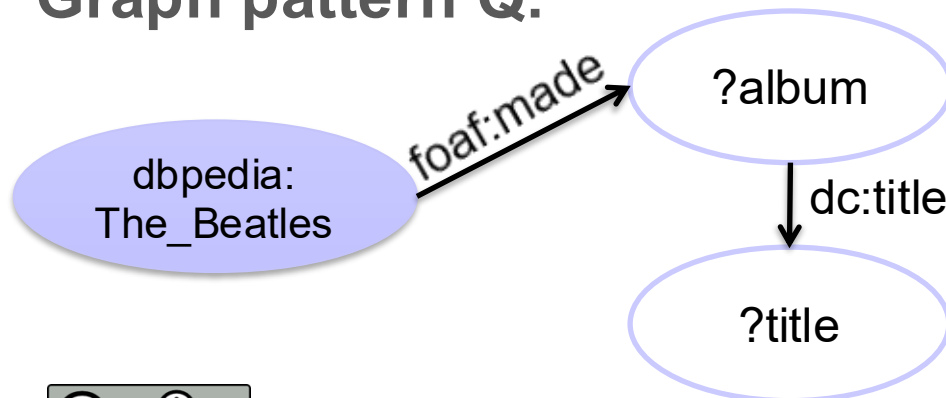
?album	?title
<http://...>	"Help!"
<http://...>	"Abbey Road"
<http://...>	"Let It Be"

SPARQL Query

Data:



Graph pattern Q:



Var(Q) = {?album, ?title}

Term(Q) = {?album, ?title, dbpedia:The_Beatles, foaf:made, dc:title}

Semantics of Graph Patterns: Key Intuitions

- Evaluation of a basic graph pattern Q over a data graph G , defined as $Q(G)$
 - The evaluation consists of successful substitutions of variables in Q with constants in G
 - A *substitution* is defined as a **mapping** from variables in Q to constants in G
 - *Successful*: once we substitute variables in Q with constants in G , **we obtain a subset of G**
 - We may have from zero to many successful substitutions

Evaluation of Basic Graph Patterns: Definition

Towards defining the results of evaluating a basic graph pattern over a data graph (following the same model), we first define a partial mapping $\mu : \mathbf{Var} \rightarrow \mathbf{Con}$ from variables to constants, whose *domain* (the set of variables for which it is defined) is denoted by $\mathbf{dom}(\mu)$. Given a basic graph pattern Q , let $\mathbf{Var}(Q)$ denote the set of all variables appearing in (some recursively nested element of) Q . We further denote by $\mu(Q)$ the image of Q under μ , meaning that any variable $v \in \mathbf{Var}(Q) \cap \mathbf{dom}(\mu)$ is replaced in Q by $\mu(v)$. Observe that when $\mathbf{Var}(Q) \subseteq \mathbf{dom}(\mu)$, then $\mu(Q)$ is a data graph (in the corresponding model of Q).

Next, we define the notion of containment between data graphs. For two directed edge-labelled graphs $G_1 = (V_1, E_1, L_1)$ and $G_2 = (V_2, E_2, L_2)$, we say that G_1 is a *sub-graph* of G_2 , denoted $G_1 \subseteq G_2$, if and only if $V_1 \subseteq V_2$, $E_1 \subseteq E_2$, and $L_1 \subseteq L_2$.³

Definition 2.7 (Evaluation of a basic graph pattern)

Let Q be a basic graph pattern and let G be a data graph (in the same model). We then define the evaluation of the basic graph pattern Q over the data graph G , denoted $Q(G)$, to be the set of mappings $Q(G) = \{\mu \mid \mu(Q) \subseteq G \text{ and } \mathbf{dom}(\mu) = \mathbf{Var}(Q)\}$.

Evaluation of Basic Graph Patterns: Definition

Towards defining the results of evaluating a basic graph pattern over a data graph (following the same model), we first define a partial mapping $\mu : \mathbf{Var} \rightarrow \mathbf{Con}$ from variables to constants, whose domain (the set of variables for which it is defined) is denoted by $\mathbf{dom}(\mu)$. Given a basic graph pattern Q , we further denote by $\mu(Q)$ the image of Q under μ , meaning that any variable $v \in \mathbf{Var}(Q) \cap \mathbf{dom}(\mu)$ is replaced in Q by $\mu(v)$. Observe that when $\mathbf{Var}(Q) \subseteq \mathbf{dom}(\mu)$, then $\mu(Q)$ is a data graph (in the context of the model).

A partial function: could be undefined for some variable

The basic graph patterns after the substitution

Next, we define the notion of containment between data graphs. Given two data graphs $G_1 = (V_1, E_1, L_1)$ and $G_2 = (V_2, E_2, L_2)$, we say that G_1 is contained in G_2 , denoted $G_1 \subseteq G_2$, if and only if $V_1 \subseteq V_2$, $E_1 \subseteq E_2$, and $L_1 \subseteq L_2$.³

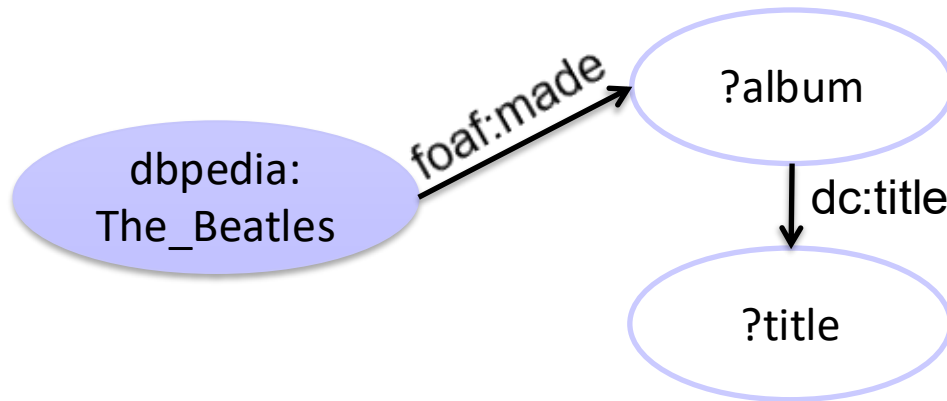
Q after the substitution is a subset of G and the variables for which the mapping is defined are exactly those in Q

Definition 2.7 (Evaluation of a basic graph pattern)

Let Q be a basic graph pattern and let G be a data graph (in the same model). We then define the evaluation of the basic graph pattern Q over the data graph G , denoted $Q(G)$, to be the set of mappings $Q(G) = \{\mu \mid \mu(Q) \subseteq G \text{ and } \mathbf{dom}(\mu) = \mathbf{Var}(Q)\}$.

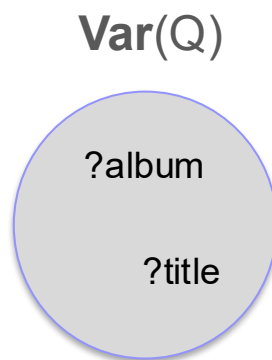
Evaluation of Basic Graph Patterns: Example

Graph pattern Q



Var(Q) = {?album, ?title}
Term(Q) = {?album, ?title, dbpedia:
The_Beatles, foaf:made, dc:title}

Mappings (functions)



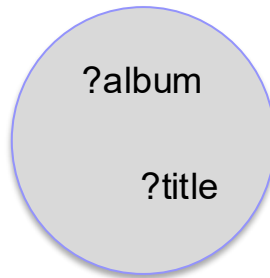
Cons(G) ... G = DBpedia



Evaluation of Basic Graph Patterns: Example

Mappings (functions)

Var(Q)



Cons(G) ... G = DBpedia



All the possible mappings for Q:

μ_1

?album $\rightarrow$ <http://...help!>
?title $\rightarrow$ Help!

μ_2

?album $\rightarrow$ <http://...Elton_John>
?title

μ_3

?album $\rightarrow$ <http://...help!>
?title $\rightarrow$ Let it be

μ_4

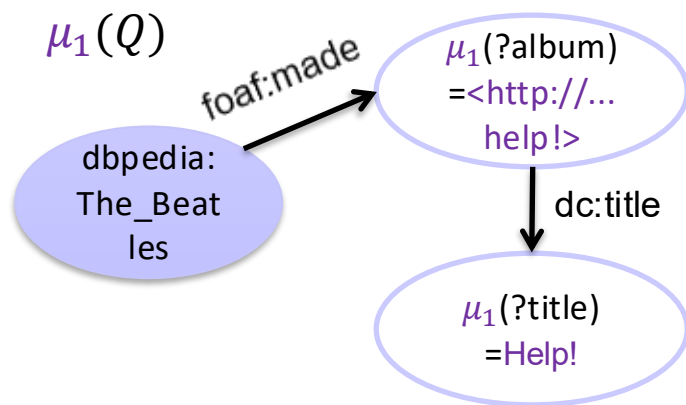
?album $\rightarrow$ <http://...it_be>
?title $\rightarrow$ Let it be

$\mu_{...}$

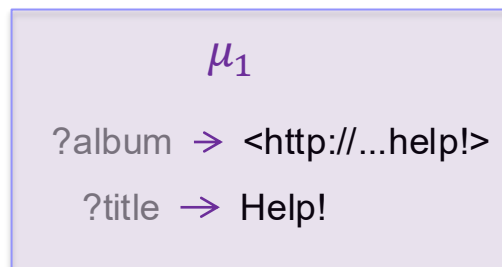
?album $\rightarrow$...
?title $\rightarrow$...

Evaluation of Basic Graph Patterns: Example

Substitution



All the possible mappings for Q:



μ_2

?album → <http://...Elton_John>
?title

μ_3

?album → <http://...help!>
?title → Let it be

μ_4

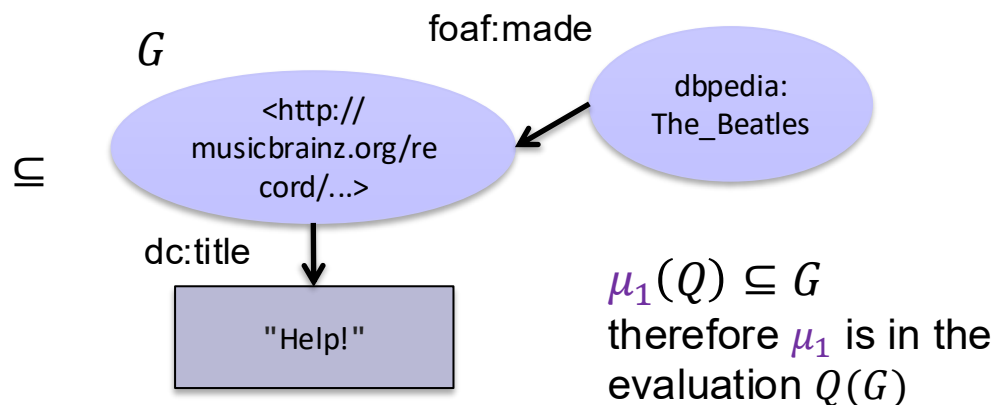
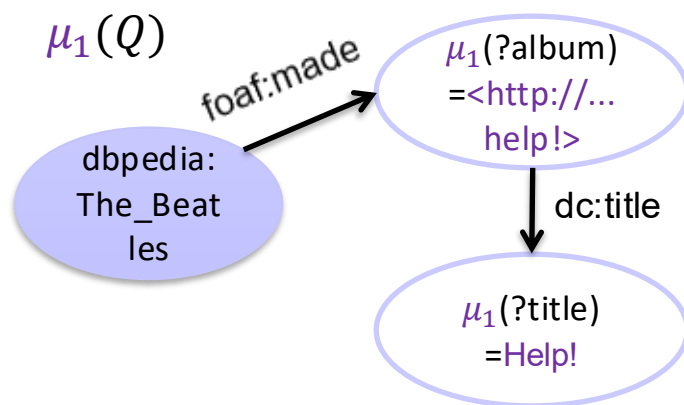
?album → <http://...it_be>
?title → Let it be

$\mu_{...}$

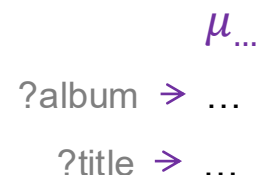
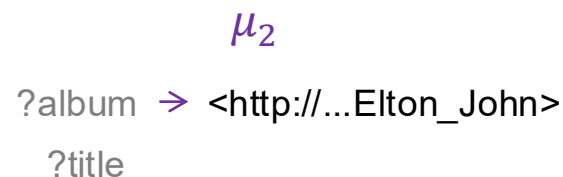
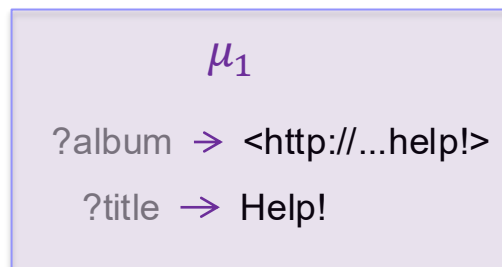
?album → ...
?title → ...

Evaluation of Basic Graph Patterns: Example

Substitution and evaluation

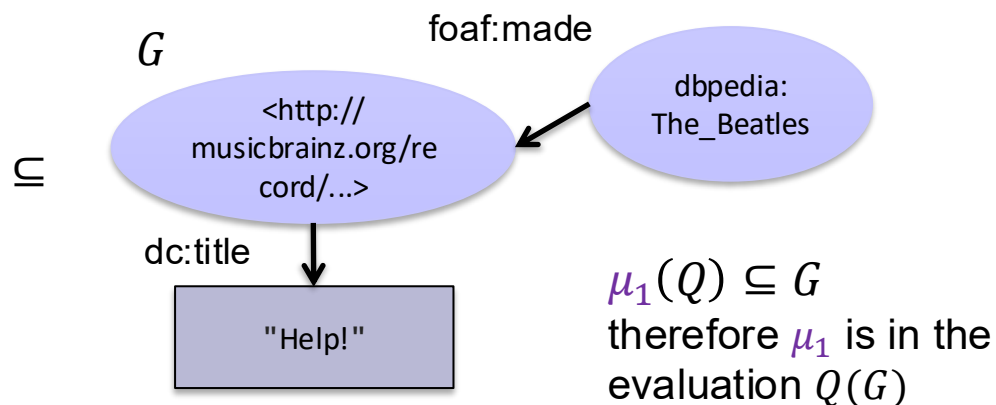
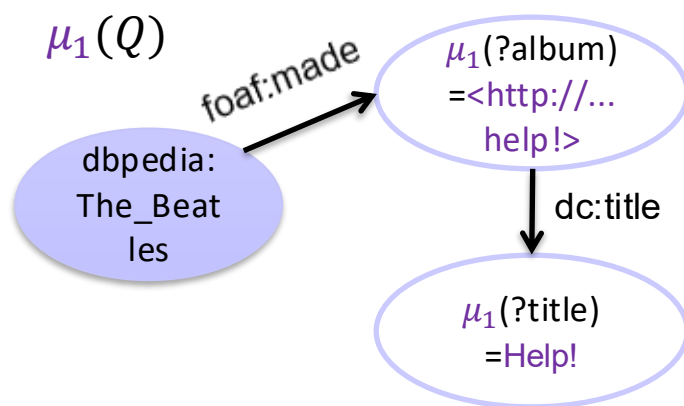


All the possible mappings for Q:



Evaluation of Basic Graph Patterns: Example

Substitution and evaluation



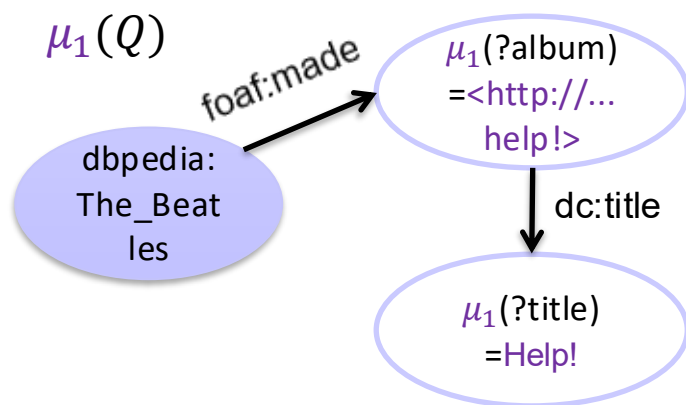
μ_1	
?album	$\rightarrow$ <http://...help!>
?title	$\rightarrow$ Help!

Mappings as rows in a table

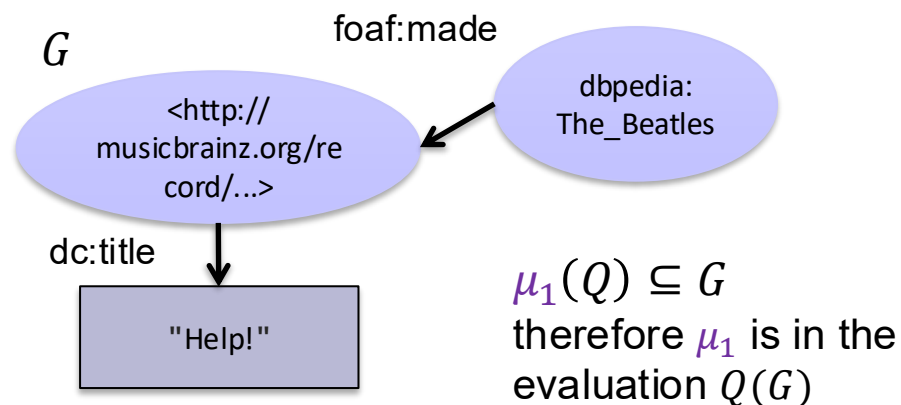
	?album	?title
$\mu_1(Q) \subseteq G$	<http://...>	"Help!"
$\mu_i(Q) \subseteq G$	<http://...>	"Abbey Road"
$\mu_j(Q) \subseteq G$	<http://...>	"Let It Be"

Evaluation of Basic Graph Patterns: Example

Substitution and evaluation



$\sqsubseteq$



Mappings as **rows** in a table

An evaluation returns a **table** (!)

$\mu_1(Q) \subseteq G$

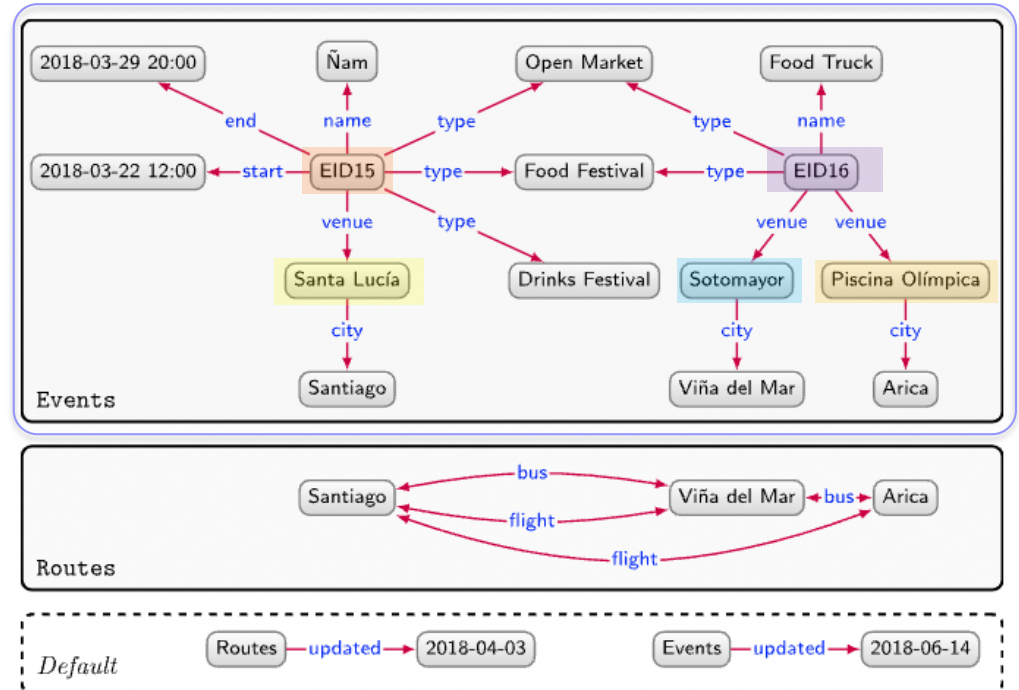
$\mu_i(Q) \subseteq G$

$\mu_j(Q) \subseteq G$

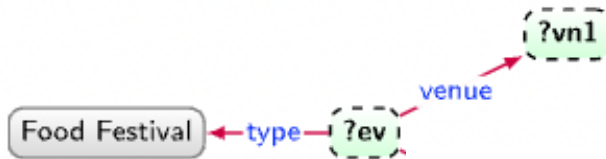
?album	?title
<http://...>	"Help!"
<http://...>	"Abbey Road"
<http://...>	"Let It Be"

Basic Graph Pattern Evaluation: Example#2

Graph



Query

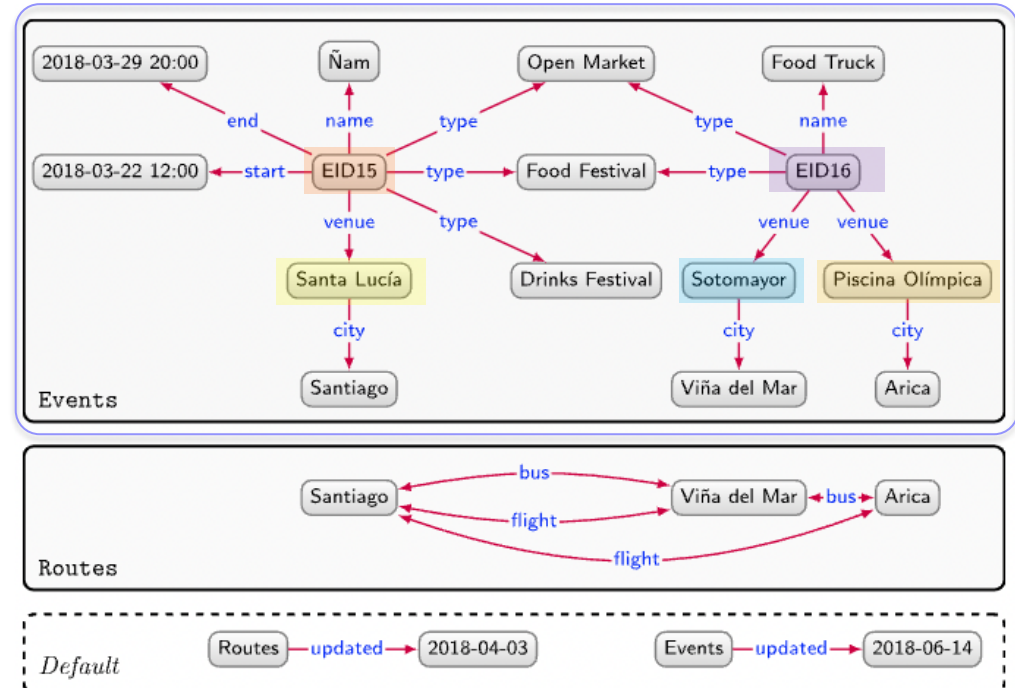


Evaluation

?ev	?vn1
EID16	Piscina Olímpica
EID16	Sotomayor
EID15	Santa Lucía

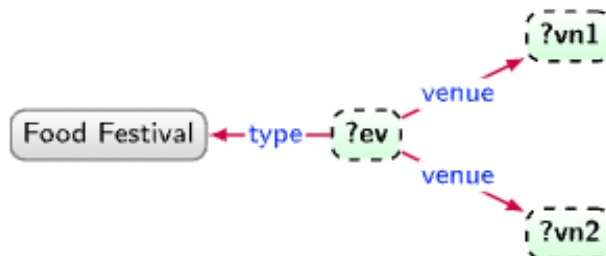
Basic Graph Pattern Evaluation: Example#2

Graph



Remark: no assumption on substitution of different variables. The mapping can be a non injective function

Query



Evaluation

?ev	?vn1	?vn2
EID16	Piscina Olímpica	Sotomayor
EID16	Sotomayor	Piscina Olímpica
EID16	Piscina Olímpica	Piscina Olímpica
EID16	Sotomayor	Sotomayor
EID15	Santa Lucía	Santa Lucía

From Basic to Complex Graph Patterns

Since evaluations of basic graph patterns return tables, we can consider complex graph patterns that specify how to manipulate tables, as in SQL

A (basic) graph pattern transforms an input graph into a table of results (as shown in Figure 2.5). We may then consider using the relational algebra to combine and/or transform such tables, thus forming more complex queries from one or more graph patterns. Recall that the relational algebra consists of unary operators that accept one input table, and binary operators that accept two input tables. Unary operators include projection (π) to output a subset of columns, selection (σ) to output a subset of rows matching a given condition, and renaming of columns (ρ). Binary operators include union ($\cup$) to merge the rows of two tables into one table, difference ($-$) to remove the rows from the first table present in the second table, and joins ($\bowtie$) to extend the rows of one table with rows from the other table that satisfy a join condition. Selection and join conditions typically include equalities ($=$), inequalities ($\leq$), negation ($\neg$), disjunction ($\vee$), etc. From these operators, we can further define other (syntactic) operators, such as intersection ($\cap$) to output rows in both tables, anti-join ($\triangleright$, aka *minus*) to output rows from the first table for which there are no join-compatible rows in the second table, left-join ($\bowtie$, aka *optional*) to perform a join but keeping rows from the first table without a compatible row in the second table, etc.

Evaluation of Complex Graph Patterns

Definition 2.8 (Complex graph pattern)

Complex graph patterns are defined recursively, as follows:

- If Q is a basic graph pattern, then Q is a complex graph pattern.
- If Q is a complex graph pattern, and $\mathcal{V} \subseteq \mathbf{Var}(Q)$, then $\pi_{\mathcal{V}}(Q)$ is a complex graph pattern.
- If Q is a complex graph pattern, and R is a selection condition with Boolean and equality connectives ($\wedge, \vee, \neg, =$), then $\sigma_R(Q)$ is a complex graph pattern.
- If both Q_1 and Q_2 are complex graph patterns, then $Q_1 \bowtie Q_2$, $Q_1 \cup Q_2$, $Q_1 - Q_2$ and $Q_1 \triangleright Q_2$ are also complex graph patterns.

Projection:
keep only a
subset of
columns

Selection: keep
only a subset
of rows that
satisfy some
conditions

Joins, unions,
...

Definition 2.9 (Complex graph pattern evaluation)

Given a complex graph pattern Q , if Q is a basic graph pattern, then $Q(G)$ is defined per Definition 2.7. Otherwise, $Q(G)$ is defined as follows:

- ! $\pi_{\mathcal{V}}(Q)(G) = \{\mu[\mathcal{V}] \mid \mu \in Q(G)\}$
- ! $\sigma_R(Q)(G) = \{\mu \mid \mu \in Q(G) \text{ and } \mu \models R\}$
- $Q_1 \bowtie Q_2(G) = \{\mu_1 \cup \mu_2 \mid \mu_1 \in Q_2(G), \mu_2 \in Q_1(G) \text{ and } \mu_1 \sim \mu_2\}$
- ! $Q_1 \cup Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ or } \mu \in Q_2(G)\}$
- $Q_1 - Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ and } \mu \notin Q_2(G)\}$
- $Q_1 \triangleright Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ and } \nexists \mu_2 \in Q_2(G) \text{ such that } \mu \sim \mu_2\}$

Evaluation of Complex Graph Patterns

Definition 2.8 (Complex graph pattern)

Complex graph patterns are defined recursively, as follows:

- If Q is a basic graph pattern, then Q is a complex graph pattern.
- If Q is a complex graph pattern, and $\mathcal{V} \subseteq \mathbf{Var}(Q)$, then $\pi_{\mathcal{V}}(Q)$ is a complex graph pattern.
- If Q is a complex graph pattern, and R is a selection condition with Boolean and equality connectives ($\wedge, \vee, \neg, =$), then $\sigma_R(Q)$ is a complex graph pattern.
- If both Q_1 and Q_2 are complex graph patterns, then $Q_1 \bowtie Q_2$, $Q_1 \cup Q_2$, $Q_1 - Q_2$ and $Q_1 \triangleright Q_2$ are also complex graph patterns.

Projection:
keep only a
subset of
columns

Selection: keep
only a subset
of rows that
satisfy some
conditions

Joins, unions,
...

Definition 2.9 (Complex graph pattern evaluation)

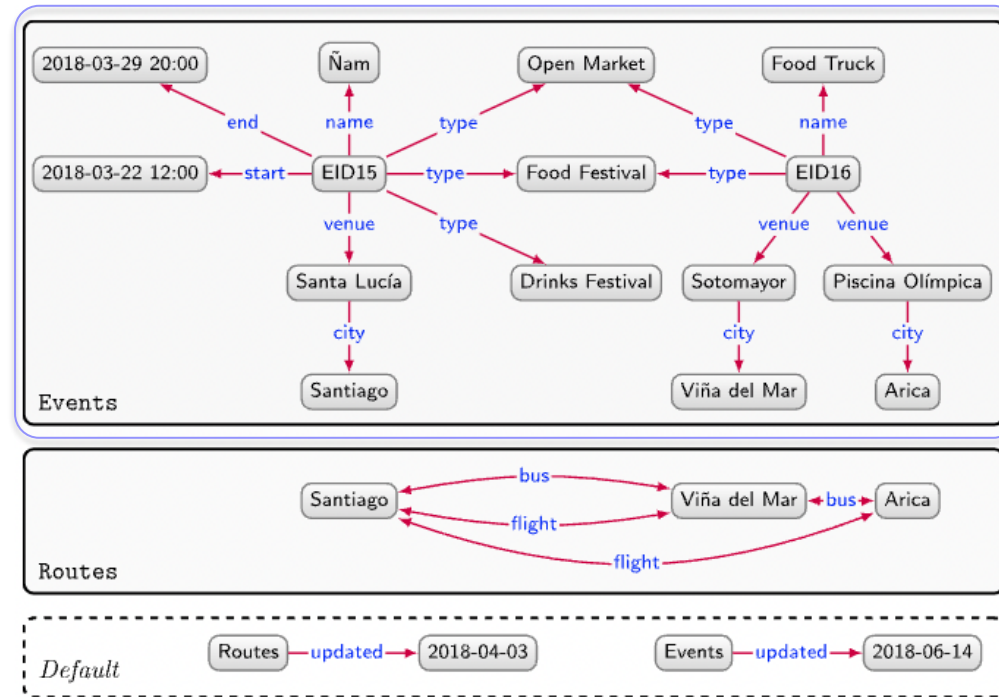
Given a complex graph pattern Q , if Q is a basic graph pattern, then $Q(G)$ is defined per Definition 2.7. Otherwise, $Q(G)$ is defined as follows:

- ! $\pi_{\mathcal{V}}(Q)(G) = \{\mu[\mathcal{V}] \mid \mu \in Q(G)\}$
- ! $\sigma_R(Q)(G) = \{\mu \mid \mu \in Q(G) \text{ and } \mu \models R\}$
- $Q_1 \bowtie Q_2(G) = \{\mu_1 \cup \mu_2 \mid \mu_1 \in Q_2(G), \mu_2 \in Q_1(G) \text{ and } \mu_1 \sim \mu_2\}$
- ! $Q_1 \cup Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ or } \mu \in Q_2(G)\}$
- $Q_1 - Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ and } \mu \notin Q_2(G)\}$
- $Q_1 \triangleright Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ and } \nexists \mu_2 \in Q_2(G) \text{ such that } \mu \sim \mu_2\}$

Complex Graph Pattern Evaluation: Projection Example

Graph

*Connections between food
festival events
(return only event names
and connections)*

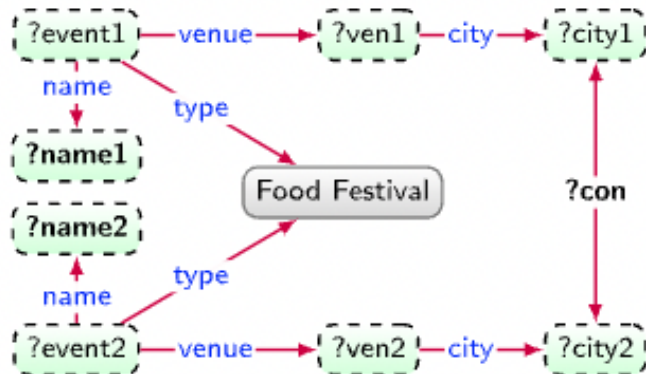
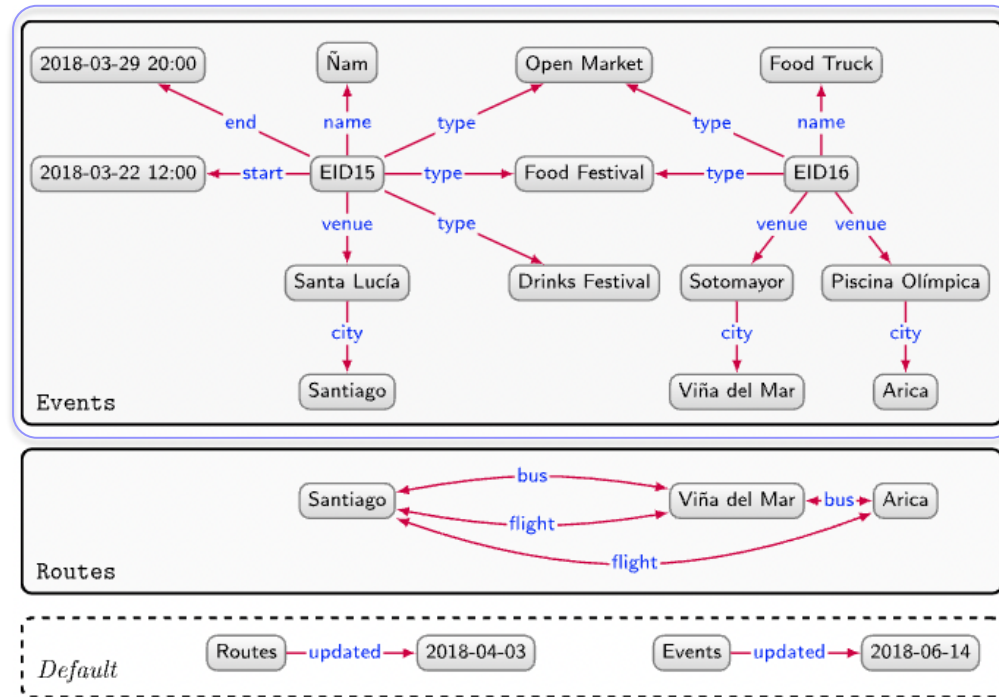


Complex Graph Pattern Evaluation: Projection Example

Graph

*Connections between food
festival events
(return only event names
and connections)*

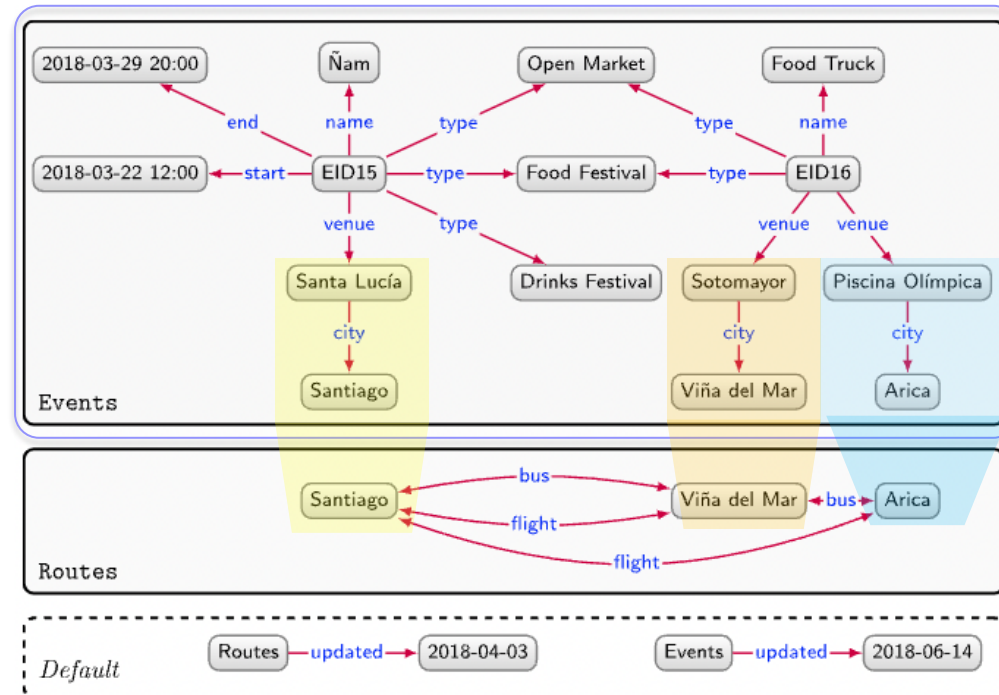
Query: **bold** variables are projected



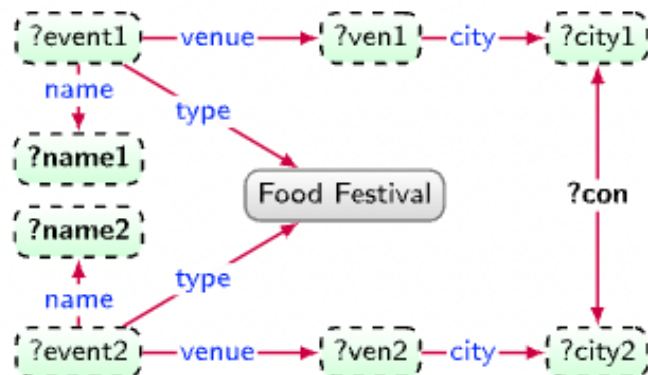
Complex Graph Pattern Evaluation: Projection Example

Graph

*Connections between food
festival events
(return only event names
and connections)*



Query: **bold** variables are projected



Evaluation

?name1	?con	?name2
Food Truck	bus	Food Truck
Food Truck	bus	Food Truck
Food Truck	bus	Ñam
Food Truck	flight	Ñam
Food Truck	flight	Ñam
Ñam	bus	Food Truck
Ñam	flight	Food Truck
Ñam	flight	Food Truck

(!) different
substitutions of
non-projected
variables

EID16,
Piscina Olimpica,
Arica

EID16,
Sotomayor, Vina
del Mar

Evaluation of Complex Graph Patterns

Definition 2.8 (Complex graph pattern)

Complex graph patterns are defined recursively, as follows:

- If Q is a basic graph pattern, then Q is a complex graph pattern.
- If Q is a complex graph pattern, and $\mathcal{V} \subseteq \mathbf{Var}(Q)$, then $\pi_{\mathcal{V}}(Q)$ is a complex graph pattern.
- If Q is a complex graph pattern, and R is a selection condition with Boolean and equality connectives ($\wedge, \vee, \neg, =$), then $\sigma_R(Q)$ is a complex graph pattern.
- If both Q_1 and Q_2 are complex graph patterns, then $Q_1 \bowtie Q_2$, $Q_1 \cup Q_2$, $Q_1 - Q_2$ and $Q_1 \triangleright Q_2$ are also complex graph patterns.

Projection:
keep only a
subset of
columns

Selection: keep
only a subset
of rows that
satisfy some
conditions

Joins, unions,
...

Definition 2.9 (Complex graph pattern evaluation)

Given a complex graph pattern Q , if Q is a basic graph pattern, then $Q(G)$ is defined per Definition 2.7. Otherwise, $Q(G)$ is defined as follows:

- ! $\pi_{\mathcal{V}}(Q)(G) = \{\mu[\mathcal{V}] \mid \mu \in Q(G)\}$
- ! $\sigma_R(Q)(G) = \{\mu \mid \mu \in Q(G) \text{ and } \mu \models R\}$
- $Q_1 \bowtie Q_2(G) = \{\mu_1 \cup \mu_2 \mid \mu_1 \in Q_2(G), \mu_2 \in Q_1(G) \text{ and } \mu_1 \sim \mu_2\}$
- ! $Q_1 \cup Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ or } \mu \in Q_2(G)\}$
- $Q_1 - Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ and } \mu \notin Q_2(G)\}$
- $Q_1 \triangleright Q_2(G) = \{\mu \mid \mu \in Q_1(G) \text{ and } \nexists \mu_2 \in Q_2(G) \text{ such that } \mu \sim \mu_2\}$

The mapping
satisfy the
condition
(FILTERS)

Mappings are
compatible, i.e.,
they map common
variables to the
same constants

Anti-join: all tuples that satisfy
Q1 and do not satisfy Q2. Only
tuples from Q1 in the results

Evaluation of Complex Graph Patterns

Definition 2.8 (Complex graph pattern)

Complex graph patterns are defined recursively, as follows:

- If Q is a basic graph pattern, then Q is a complex graph pattern.
- If Q is a complex graph pattern, and $\mathcal{V} \subseteq \mathbf{Var}(Q)$, then $\sigma_{\mathcal{V}}(Q)$ is a complex graph pattern.
- If Q is a complex graph pattern, and R is a selection of connectives ($\wedge, \vee, \neg, =$), then $\sigma_R(Q)$ is a complex graph pattern.
- If both Q_1 and Q_2 are complex graph patterns, then $Q_1 \triangleright Q_2$ are also complex graph patterns.

In practice: natural join of the two result tables

By combining different operators we can define left-join, which is useful to define OPTIONAL clauses, i.e., data to add to the results only if available (example later)

Definition 2.9 (Complex graph pattern evaluation)

Given a complex graph pattern Q , if Q is a basic graph pattern, $Q(G) = \{\mu \mid \mu \models Q\}$.

Otherwise, $Q(G)$ is defined as follows:

$$\begin{aligned} \pi_{\mathcal{V}}(Q)(G) &= \{\mu[\mathcal{V}] \mid \mu \in Q(G)\} \\ \sigma_R(Q)(G) &= \{\mu \mid \mu \in Q(G) \text{ and } \mu \models R\} \\ Q_1 \bowtie Q_2(G) &= \{\mu_1 \cup \mu_2 \mid \mu_1 \in Q_2(G), \mu_2 \in Q_1(G) \text{ and } \mu_1 \sim \mu_2\} \\ Q_1 \cup Q_2(G) &= \{\mu \mid \mu \in Q_1(G) \text{ or } \mu \in Q_2(G)\} \\ Q_1 - Q_2(G) &= \{\mu \mid \mu \in Q_1(G) \text{ and } \mu \notin Q_2(G)\} \\ Q_1 \triangleright Q_2(G) &= \{\mu \mid \mu \in Q_1(G) \text{ and } \nexists \mu_2 \in Q_2(G) \text{ such that } \mu \sim \mu_2\} \end{aligned}$$

Mappings are compatible, i.e., they map common variables to the same constants



SPARQL

Query RDF data

03 – Path Queries: Syntax and Semantics

Path Queries #1

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>

9.2 Examples

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

Alternatives: Match one or both possibilities

```
{ :book1 dc:title|rdfs:label ?displayString }
```

which could have written:

```
{ :book1 <http://purl.org/dc/elements/1.1/title> | <http://www.w3.org/2000/01/rdf-schema#label> ?displayString }
```

Sequence: Find the name of any people that Alice knows.

```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows/foaf:name ?name .  
}
```

Sequence: Find the names of people 2 "foaf:knows" links away.

```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows/foaf:knows/foaf:name ?name .  
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

Inverse Property Paths: These two are the same query: the second is just reversing the property direction which swaps the roles of subject and object.

```
{ ?x foaf:mbox <mailto:alice@example> }
```

```
{ <mailto:alice@example> ^foaf:mbox ?x }
```

Inverse Path Sequence: Find all the people who know someone ?x knows.

```
{  
  ?x foaf:knows/^foaf:knows ?y .  
  FILTER(?x != ?y)  
}
```

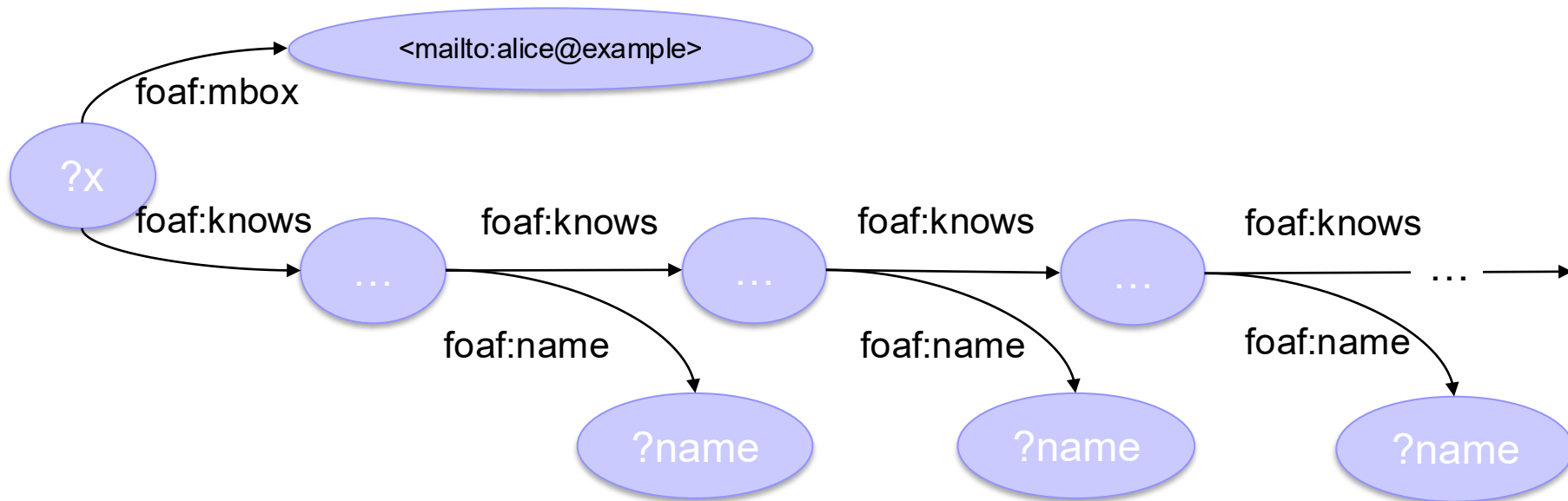
Arbitrary length match: Find the names of all the people that can be reached from Alice by foaf:knows:

```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows+/foaf:name ?name .  
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>



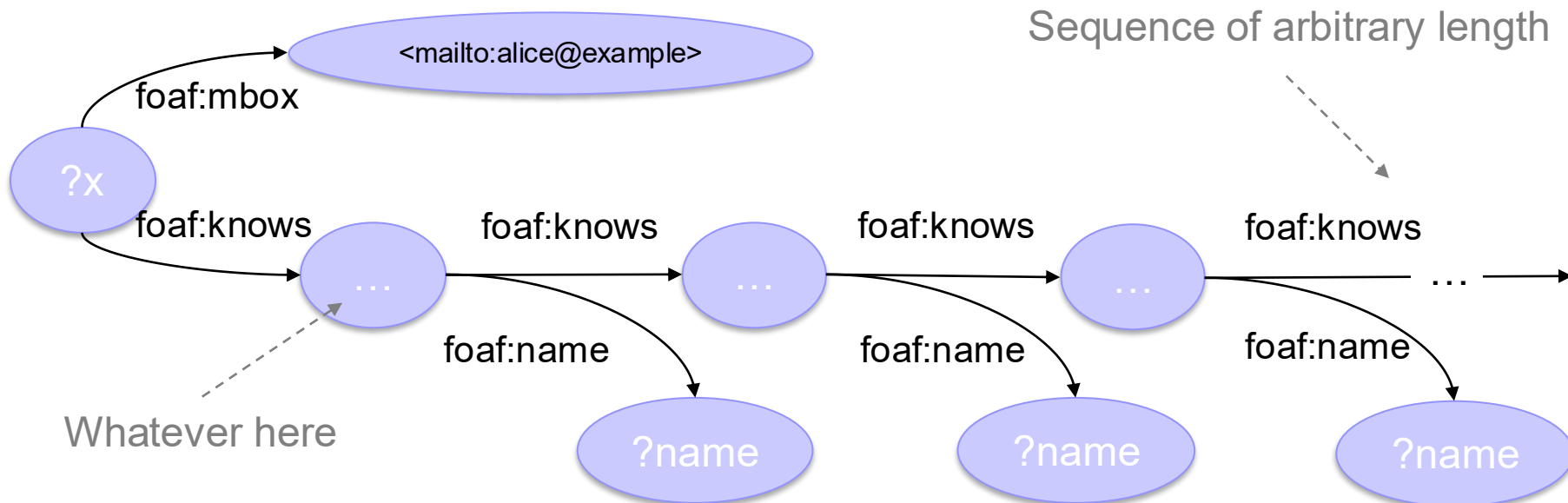
Arbitrary length match: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{
  ?x foaf:mbox <mailto:alice@example> .
  ?x foaf:knows+/foaf:name ?name .
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertices are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>



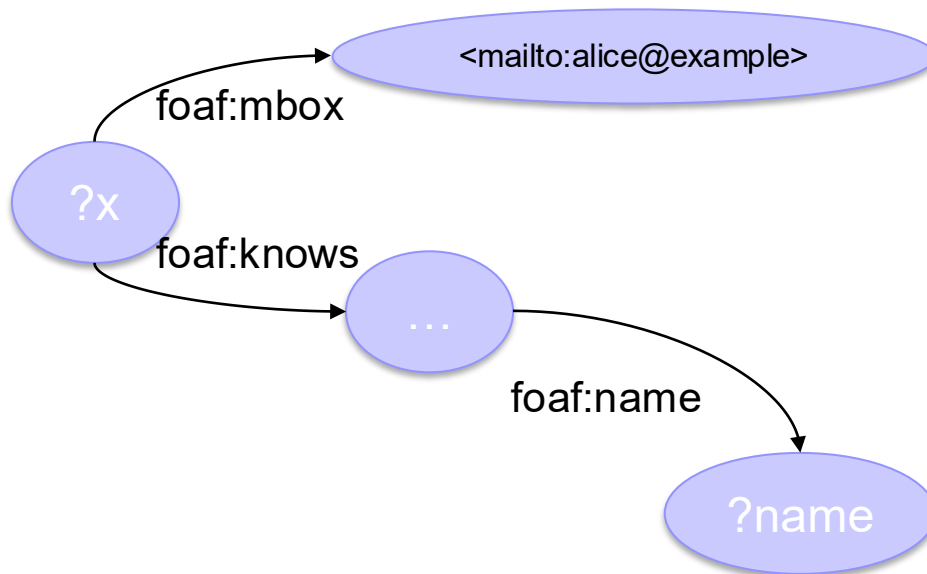
Arbitrary length match: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows+/foaf:name ?name .  
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>



More precisely: as if we make n queries, adding a `foaf:knows` edge at every time

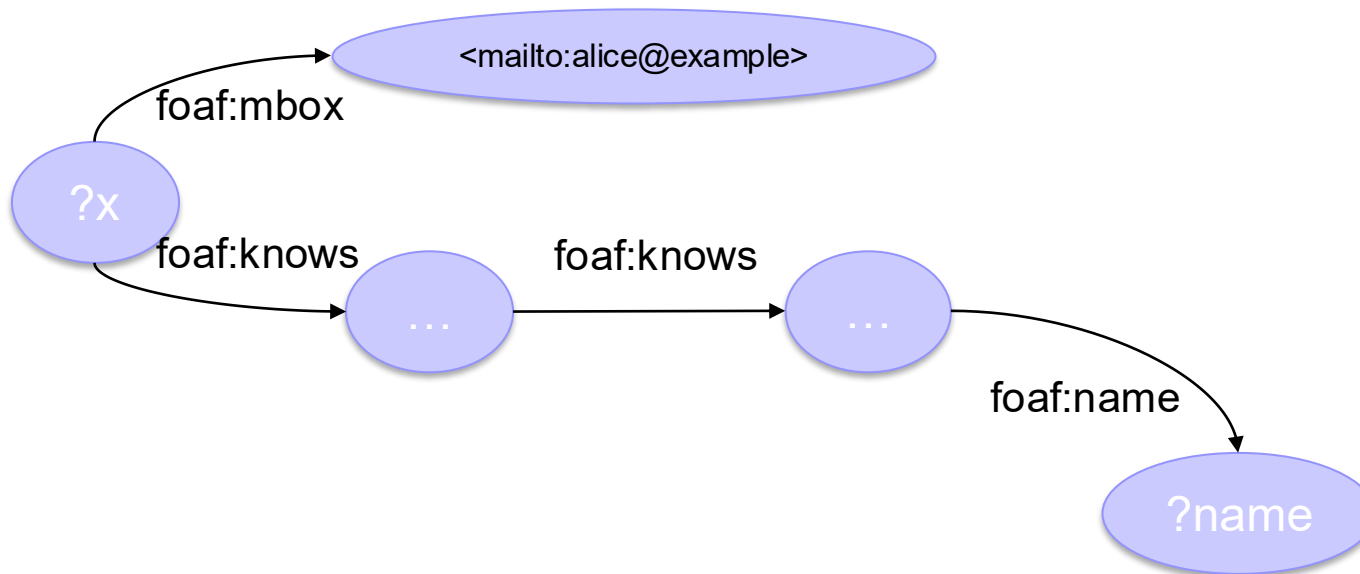
Arbitrary length match: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{
  ?x foaf:mbox <mailto:alice@example> .
  ?x foaf:knows+/foaf:name ?name .
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>



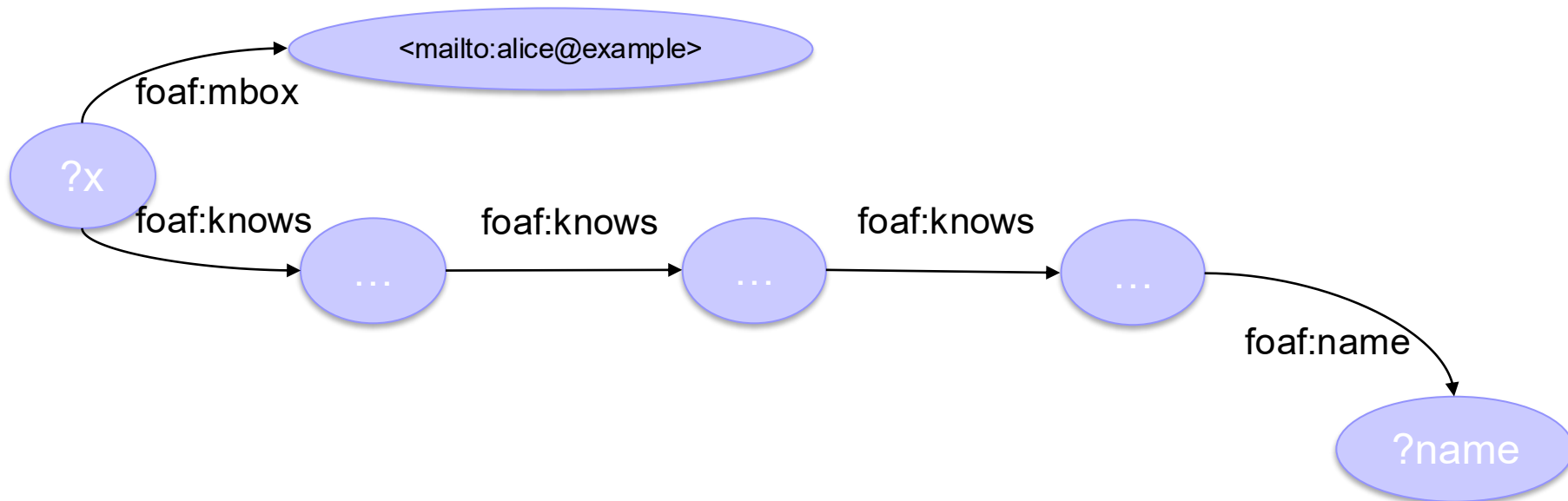
Arbitrary length match: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{
  ?x foaf:mbox <mailto:alice@example> .
  ?x foaf:knows+/foaf:name ?name .
}
```


Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>



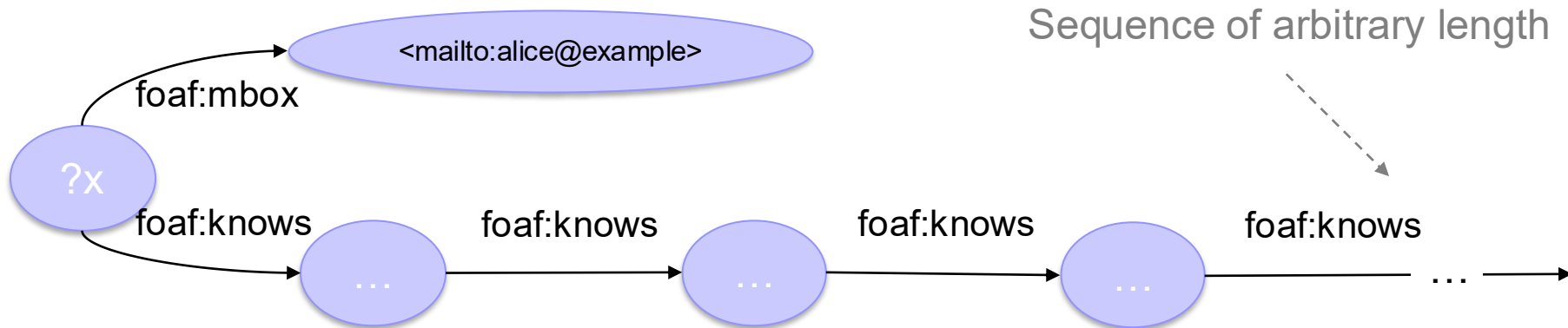
Arbitrary length match: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{
  ?x foaf:mbox <mailto:alice@example> .
  ?x foaf:knows+/foaf:name ?name .
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>



Arbitrary length match: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows+/foaf:name ?name .  
}
```

Path Queries #2

Variables cannot be used as part of the path itself
Vertexes are returned when the paths are evaluated

- Check: <https://www.w3.org/TR/sparql11-query/#propertypaths>

Homework: draw the other path queries

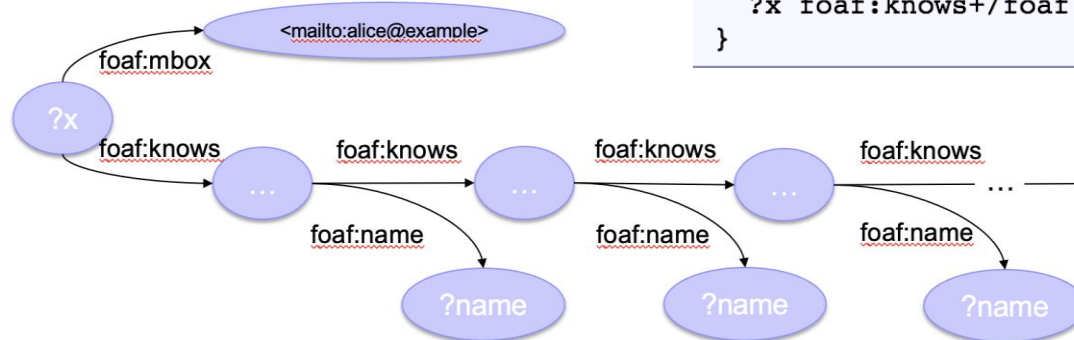
```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows+/foaf:name ?name .  
}
```

Semantics of Path Queries: Key Intuitions

- **Path expressions** and their evaluation
 - A **path expression** specifies a way to traverse the data graph, i.e., some paths
 - The **evaluation of a path** consists in all **pairs of nodes** that satisfy the expression, i.e., connected through the specified path
- **Regular path queries** and their evaluation
 - A path expression whose nodes can be **variables**
 - The **evaluation of a path query** consists in **substitutions of variables** that produce successful paths (non-empty path evaluations), similarly as for graph pattern queries
 - In fact, we can mix path queries with graph patterns into navigational graph patterns (join of their evaluation)

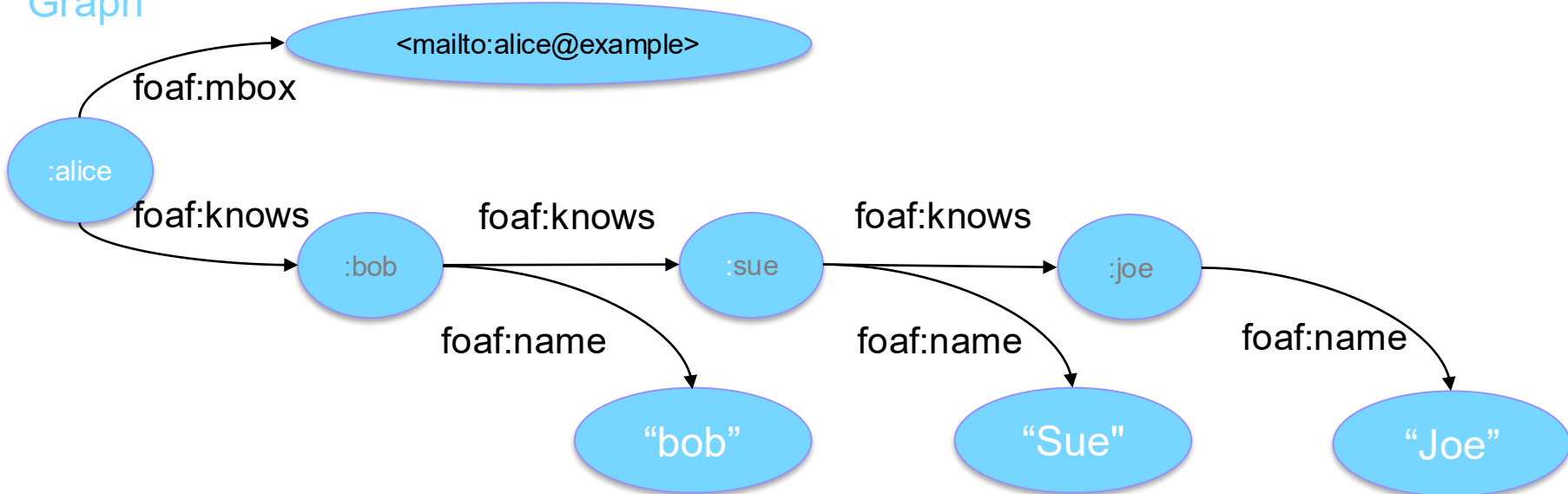
Intuition: Reg. Path Query Example

Path query



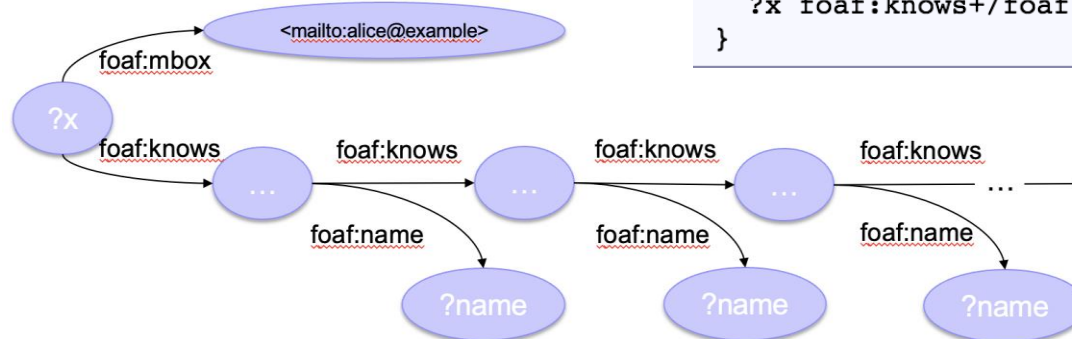
```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows+/foaf:name ?name .  
}
```

Graph



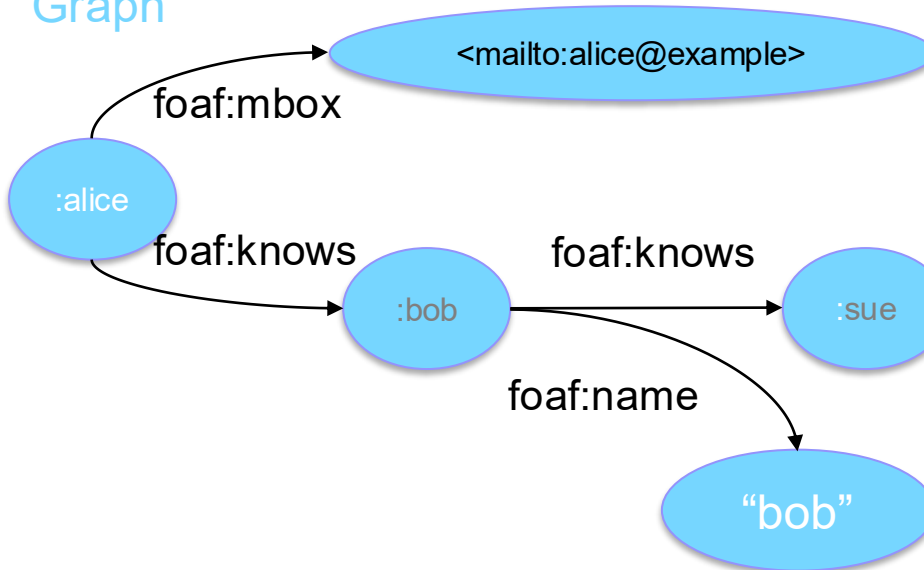
Intuition: Reg. Path Query Example

Path query



```
{
  ?x foaf:mbox <mailto:alice@example> .
  ?x foaf:knows+/foaf:name ?name .
}
```

Graph



Results

?x	?name
:alice	"bob"
:alice	"Sue"
:alice	"Joe"

Evaluation of Path Expressions

SKIP / FAST
No variables,
just path expressions
over a graph

Definition 2.10 (Path expression)

A constant (edge label) c is a path expression. Furthermore, if r , r_1 and r_2 are path expressions, then:

- r^- (inverse) and r^* (Kleene star) are path expressions.
- $r_1 \cdot r_2$ (concatenation) and $r_1 \mid r_2$ (disjunction) are path expressions.

r^* = zero or more paths
Why "zero"... in the hidden slide!

We now define the evaluation of a path expression on a directed-edge labelled graph under the SPARQL 1.1-style semantics whereby the endpoints (pairs of start and end nodes) of the path are returned [Harris et al., 2013].

Definition 2.11 (Path evaluation (directed edge-labelled graph))

Given a directed edge-labelled graph $G = (V, E, L)$ and a path expression r , we define the evaluation of r over G , denoted $r[G]$, as follows:

$$\begin{aligned} r[G] &= \{(u, v) \mid (u, r, v) \in E\} \text{ (for } r \in \mathbf{Con}) \\ r^-[G] &= \{(u, v) \mid (v, u) \in r[G]\} \\ r_1 \mid r_2[G] &= r_1[G] \cup r_2[G] \\ r_1 \cdot r_2[G] &= \{(u, v) \mid \exists w \in V : (u, w) \in r_1[G] \text{ and } (w, v) \in r_2[G]\} \\ r^*[G] &= \{(u, u) \mid u \in V\} \cup \bigcup_{n \in \mathbb{N}^+} r^n[G] \end{aligned}$$

where by r^n we denote the n^{th} -concatenation of r (e.g., $r^3 = r \cdot r \cdot r$).

Intuition: Reg. Path Expression Example

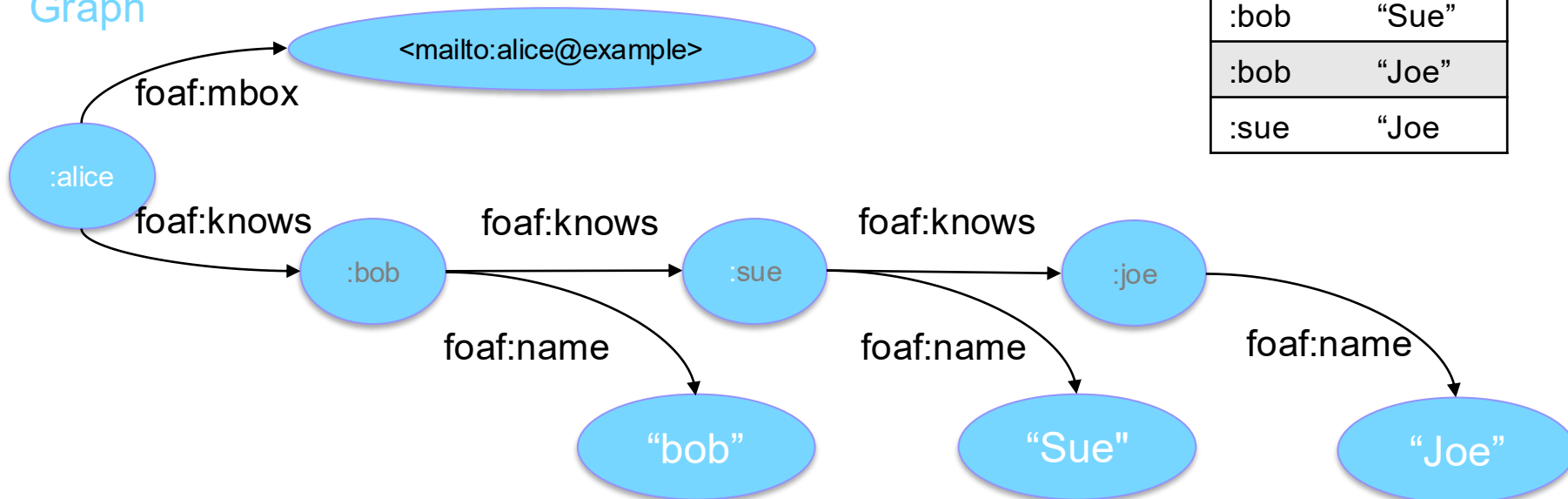
Path expression

foaf:knows+/foaf:name

Results

:alice	"bob"
:alice	"Sue"
:alice	"Joe"
:bob	"Sue"
:bob	"Joe"
:sue	"Joe"

Graph



Intuition: Reg. Path Expression Example

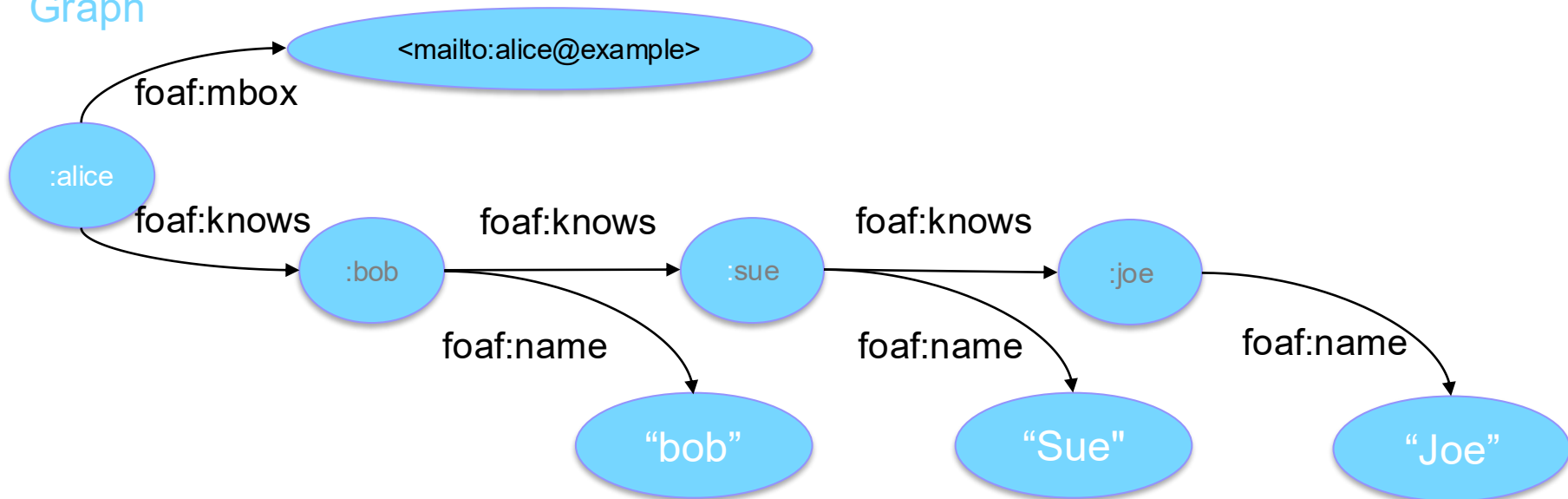
Path expression

foaf:knows/foaf:knows/foaf:name

Results

:alice	"Sue"
:bob	"Joe"

Graph



Intuition: Reg. Path Expression Example

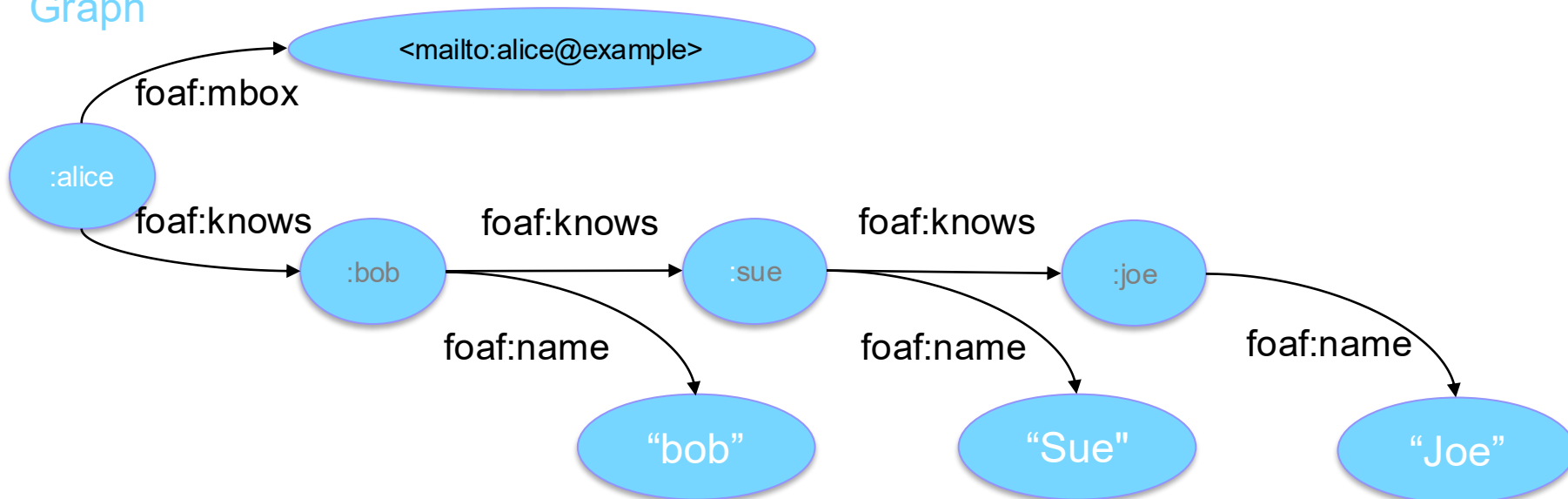
Path expression

foaf:knows/foaf:name

Results

:alice	"bob"
:bob	"Sue"
:sue	"Joe"

Graph



Evaluation of Path Queries

Regular path queries
= path expressions
+ variables
SKIP / FAST

Definition 2.12 (Regular path query)

A regular path query is a triple (x, r, y) where $x, y \in \mathbf{Con} \cup \mathbf{Var}$ and r is a path expression.

Definition 2.13 (Regular path query evaluation)

Let G denote a directed edge-labelled graph, $c, c_1, c_2 \in \mathbf{Con}$ denote constants and $z, z_1, z_2 \in \mathbf{Var}$ denote variables. Then the evaluation of a regular path query is defined as follows:

$$\begin{aligned}(c_1, r, c_2)(G) &= \{\mu_\emptyset \mid (c_1, c_2) \in r[G]\} \\ (c, r, z)(G) &= \{\mu \mid \mathbf{dom}(\mu) = \{z\} \text{ and } (c, \mu(z)) \in r[G]\} \\ (z, r, c)(G) &= \{\mu \mid \mathbf{dom}(\mu) = \{z\} \text{ and } (\mu(z), c) \in r[G]\} \\ (z_1, r, z_2)(G) &= \{\mu \mid \mathbf{dom}(\mu) = \{z_1, z_2\} \text{ and } (\mu(z_1), \mu(z_2)) \in r[G]\}\end{aligned}$$

where $\mu_\emptyset$ denotes the empty mapping such that $\mathbf{dom}(\mu) = \emptyset$ (the join identity).

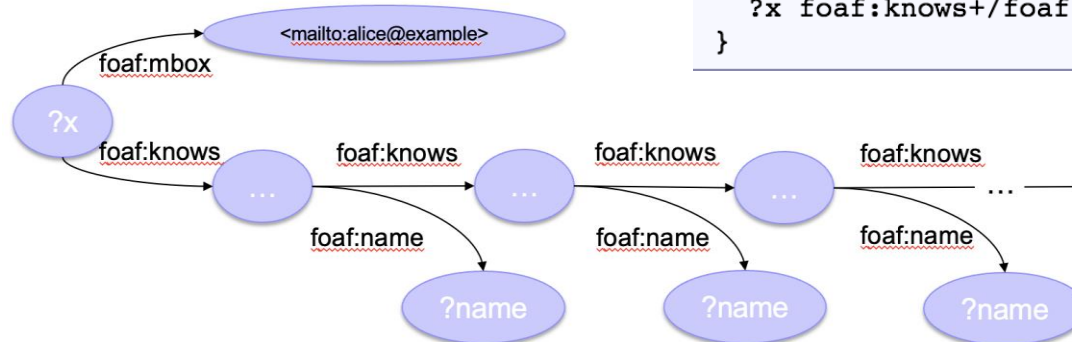
Definition 2.14 (Navigational graph pattern)

If Q is a basic graph pattern, then Q is a navigational graph pattern. If Q is a navigational graph pattern and (x, r, y) is a regular path query, then $Q \bowtie (x, r, y)$ is a navigational graph pattern.

Navigational
graph patterns:
Graph patterns
combined with
path queries
via join

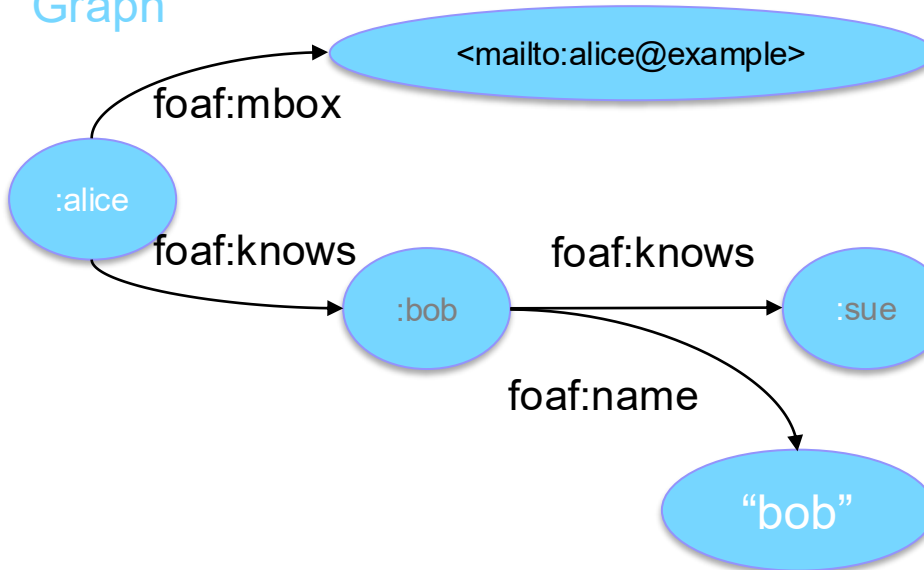
Intuition: Reg. Path Query Example

Path query



```
{  
  ?x foaf:mbox <mailto:alice@example> .  
  ?x foaf:knows+/foaf:name ?name .  
}
```

Graph



Results

?x	?name
:alice	"bob"
:alice	"Sue"
:alice	"Joe"



SPARQL

Query RDF data

04 – Query Forms and More Examples

SPARQL supports different query forms:

- **ASK** tests whether or not a query pattern has a solution. Returns yes/no
- **SELECT** returns variables and their bindings directly
- **CONSTRUCT** returns a single RDF graph specified by a graph template
- **DESCRIBE** returns a single RDF graph containing RDF data about resource

Query Form: ASK

- Namespaces are added with the ‘PREFIX’ directive
- Statement patterns that make up the graph are specified between brackets (“{ }”)

Query: *Is Paul McCartney member of ‘The Beatles’?*

```
PREFIX dbpedia: <http://dbpedia.org/resource/>  
PREFIX dbpedia-ont: <http://dbpedia.org/ontology/>  
PREFIX mo: http://purl.org/ontology/mo/  
ASK WHERE { dbpedia:The_Beatles mo:member  
             dbpedia:Paul_McCartney.}
```

Results:

true

Query: *Is Elvis Presley member of ‘The Beatles’?*

```
PREFIX dbpedia: <http://dbpedia.org/resource/>  
PREFIX dbpedia-ont: <http://dbpedia.org/ontology/>  
PREFIX mo: http://purl.org/ontology/mo/  
ASK WHERE { dbpedia:The_Beatles mo:member  
             dbpedia:Elvis_Presley.}
```

Results:

false

Query Form: SELECT

- The solution modifier **projection** nominates which components of the matches should be returned
- “*” means all components should be returned

Query: *What albums and tracks did ‘The Beatles’ make?*

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>
SELECT ?album_name ?track_title
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album dc:title ?album_name ;
           mo:track ?track .
    ?track dc:title ?track_title .}
```


Query Form: SELECT (2)

Filter expressions

- Different types of filters and functions may be used

Filter: Comparison and logical operators

Query: *Retrieve the albums and tracks recorded by 'The Beatles', where the duration of the song is more than 300 secs. and no longer than 400 secs.*

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>
SELECT ?album_name ?track_title ?duration
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album dc:title ?album_name ;
           mo:track ?track .
    ?track dc:title ?track_title ;
           mo:duration ?duration;
    FILTER (?duration>300000 && ?duration<400000) }
```

Query Form: SELECT (3)

Optional patterns

When part of graph pattern is not mandatory

Query: *Retrieve the albums and tracks recorded by 'The Beatles', add duration of each track if specified*

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
PREFIX mo: <http://purl.org/ontology/mo/>
SELECT ?album_name ?track_title ?duration
WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album dc:title ?album_name ;
           mo:track ?track .
    ?track dc:title ?track_title .
    OPTIONAL
        {?track mo:duration ?duration .}
}
```

Equivalent to left-join
between the two
result tables

Query Form: CONSTRUCT

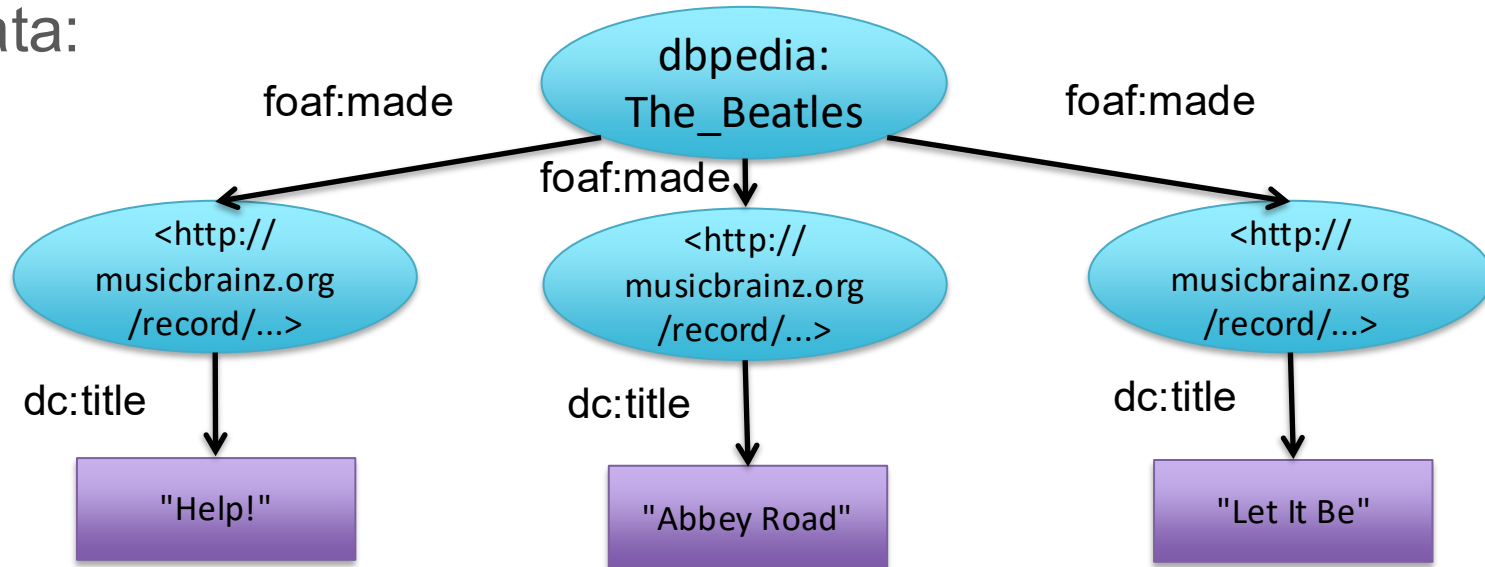
- **CONSTRUCT WHERE:** In order to query for a subgraph, without change, it is no longer necessary to repeat the graph pattern in the template

Example:

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX mo: <http://purl.org/ontology/mo/>
PREFIX dc: <http://purl.org/dc/elements/1.1/>
CONSTRUCT WHERE {
    dbpedia:The_Beatles foaf:made ?album .
    ?album mo:track ?track .}
```

Query Form: CONSTRUCT (2a)

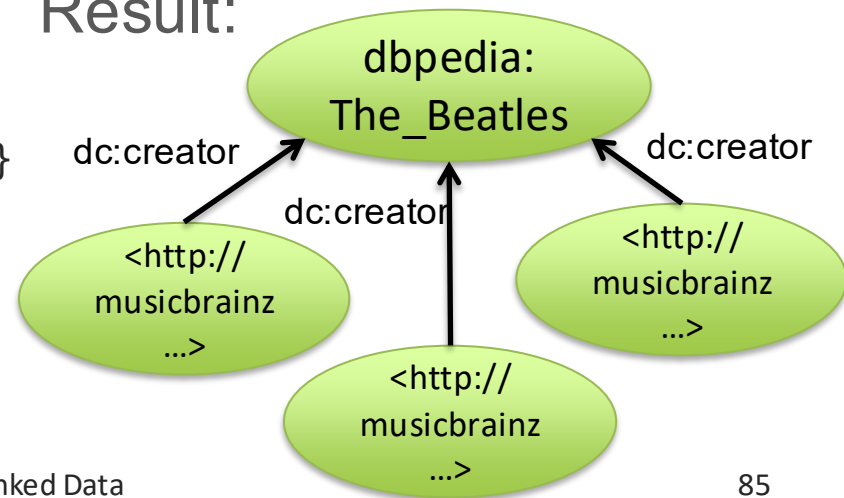
Data:



Query:

```
CONSTRUCT {  
  ?album dc:creator dbpedia:The_Beatles .}  
WHERE {  
  dbpedia:The_Beatles foaf:made ?album .}
```

Result:



SPARQL Query Formulation Exercises

- You can query DBpedia through a web interface at: <http://dbpedia.org/sparql>
- How to Know What to Query?
 - After this lesson:
 - Understand the vocabulary (properties and classes) by exploring DBpedia online (see previous slide) – [after this lesson]
 - Use explorative queries (e.g., search for all properties used with subjects of type `dbo:City`)
 - After the lessons on ontologies:
 - Look at DBpedia ontology using Protegé



SKIP / FAST

Query and/or Reasoning

05 –SPARQL & Reasoning

SPARQL Evolution

SKIP / FAST

- It used to be only a query language
- Now the protocol can support reasoning (if the SPARQL engine has a reasoner)
- We will discuss what reasoning is in the next lessons

SPARQL 1.1:

Entailment Regimes

- SPARQL 1.0 was defined only for simple entailment (pattern matching)
- SPARQL 1.1 is extended with entailment regimes other than simple entailment:
 - RDF entailment
 - **RDFS entailment**
 - D-Entailment
 - OWL RL entailment
 - OWL Full entailment
 - OWL 2 DL, EL, and QL entailment
 - RIF entailment

Entailment: ~ “implication”, ~ inference
REMARK: the triple store serving the SPARQL endpoint must support the selected entailment regime (cont'd: different stores support different entailment regimes)

Source: <http://www.w3.org/TR/rdf-mt/#RDFSRules>

SPARQL: Wrap-up

- SPARQL is inspired by SQL
- Different query forms: ASK, SELECT, CONSTRUCT, DEFINE
- The evaluation of queries return tables, pairs of nodes in the case of path queries
 - SELECT returns tables, even when we include path expressions
 - Can mix graph patterns and path queries
- CONSTRUCT return graphs and can be used for data manipulation